

claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_1c5f1f5c-9ba\agent-a44e407ce1924903a.jsonl`
- SHA-256: `8ecb0e05660de572c89ed1faf56c575f314751f5e20519f970b47e4e9fe923a1`
- Source modified: `2026-06-20T08:02:49+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_1c5f1f5c-9ba`
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`
```

Transcript

1. user (2026-06-20T08:00:25.363Z)

You are reading the badminton-highlight-indexer repo (your cwd). Map the rally-index / candidate-segments DATA CONTRACT that an interactive per-rally-selection UI would consume. Investigate and report ONLY what the code actually does (quote path:line):

1. DB schema for persisted rally/candidate segments ? read backend/infrastructure/database.py (the PRAGMA user_version migration ladder) and wherever segments are written/read. Table name(s), every column + type. Look for: start time, end time, duration, video_id, game/set index, score, detector key, confidence, label/correction.
2. Per-rally fields available TODAY vs fields that would need detection that does NOT exist yet (shot type, player identity, score).
3. How a reel is composed/stitched from a chosen set of segments ? find the ffmpeg/stitch path (e.g. backend/pipeline or services). What inputs does it take (list of intervals? segment ids? a video_id + selection?) and what does it output (single MP4 + summary)?
4. Playback: is there a low-res proxy video the UI could scrub? Are extracted frames/thumbnails available per rally? How would the UI show a rally clip?

Be exhaustive and precise. Do NOT invent fields or endpoints ? if something is absent, say so explicitly in notes. Return the schema.

2. assistant (2026-06-20T08:00:27.850Z)

I'll map the rally-index / candidate-segments data contract. Let me start by reading the key files mentioned in the project memory.

3. user (2026-06-20T08:00:32.721Z)

Your configuration specifies to merge with the ref 'refs/heads/docs/court-floor-rally-end' from the remote, but no such ref was fetched.

4. user (2026-06-20T08:00:33.192Z)

```
1      import sqlite3
2      import json
3      import logging
4      from contextlib import contextmanager
5      from typing import List, Dict, Any, Optional, Tuple
6
7      logger = logging.getLogger(__name__)
8
9      class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
```

```

16         """Transaction scope that also CLOSES the connection.
17
18         ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19         never closes, so every call leaked its connection to GC (lingering file
20         handles on the WAL sidecars under Windows). Internal methods use this.
21         """
22         conn = self._get_connection()
23         try:
24             with conn:
25                 yield conn
26         finally:
27             conn.close()
28
29     def _get_connection(self):
30         # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31         # adds concurrent writers (a swallowed 'database is locked' silently drops
32         segments).
33         conn = sqlite3.connect(self.db_path, timeout=30.0)
34         conn.row_factory = sqlite3.Row
35         # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
36         conn.execute("PRAGMA foreign_keys = ON")
37         # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
38         conn.execute("PRAGMA busy_timeout = 30000")
39         try:
40             conn.execute("PRAGMA journal_mode = WAL")
41         except sqlite3.OperationalError:
42             pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
43         return conn
44
45     def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
46         """Run ONE bool-returning single-statement write inside a connection scope.
47
48         Captures the connect/commit/except-logger.error/return-False boilerplate shared
49         by
50         the bool-returning single-statement writers. ``error_msg`` is the caller's exact
51         per-method message (already formatted with its own ids) ? logged verbatim on a
52         swallowed write fault so each writer keeps its specific log text. Returns True
53         on
54         success, False (after logging ``error_msg``) on any exception."""
55         try:
56             with self._connect() as conn:
57                 conn.cursor().execute(sql, params)
58                 conn.commit()
59             return True
60         except Exception as e:
61             logger.error(f"{error_msg}: {e}")
62             return False
63
64     @staticmethod
65     def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
66         """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runably.
67
68         A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
69         transaction, so an interrupted v3/v4 block can leave the column added while
70         ``user_version`` stays un-bumped ? and the next open would re-run the same
71         ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
72         future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
73         Swallow
74         ONLY that specific error so the ladder stays crash-safe, matching the re-
75         runnable
76
77         ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
78         """
79         try:
80             cursor.execute(ddl)
81         except sqlite3.OperationalError as exc:

```

```

76         if "duplicate column name" not in str(exc).lower():
77             raise
78
79     def _init_db(self):
80         """Initializes tables for videos, play_segments, and events."""
81         with self._connect() as conn:
82             cursor = conn.cursor()
83
84             self._create_base_tables(cursor)
85
86             # Versioned migrations live here. PRAGMA user_version is the schema
87             # contract ? bump it for every change and add an ordered `if version < N`
88             # block below. Tests assert the expected version, so accidental drift fails
89             version = cursor.execute("PRAGMA user_version").fetchone()[0]
90
91             if version < 1:
92                 self._migrate_to_v1(cursor)
93
94             if version < 2:
95                 self._migrate_to_v2(cursor)
96
97             if version < 3:
98                 self._migrate_to_v3(cursor)
99
100             if version < 4:
101                 self._migrate_to_v4(cursor)
102
103             if version < 5:
104                 self._migrate_to_v5(cursor)
105             # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
106             user_version = 6
107
108             conn.commit()
109
110     def _create_base_tables(self, cursor):
111         """Create the videos / play_segments / events base tables (idempotent)."""
112         # Videos Table
113         cursor.execute("""
114             CREATE TABLE IF NOT EXISTS videos (
115                 id TEXT PRIMARY KEY,
116                 filepath TEXT NOT NULL,
117                 processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
118                 status TEXT NOT NULL,
119                 validation_results TEXT
120             )
121         """)
122
123         # Play Segments Table (Extensible metadata JSON)
124         cursor.execute("""
125             CREATE TABLE IF NOT EXISTS play_segments (
126                 id INTEGER PRIMARY KEY AUTOINCREMENT,
127                 video_id TEXT NOT NULL,
128                 start_time REAL NOT NULL,
129                 end_time REAL NOT NULL,
130                 duration REAL NOT NULL,
131                 server TEXT,
132                 receiver TEXT,
133                 metadata TEXT,
134                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
135             )
136         """)
137
138         # Events Table
139         cursor.execute("""

```

```

139         CREATE TABLE IF NOT EXISTS events (
140             id INTEGER PRIMARY KEY AUTOINCREMENT,
141             segment_id INTEGER NOT NULL,
142             timestamp REAL NOT NULL,
143             type TEXT NOT NULL,
144             player TEXT,
145             details TEXT,
146             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
147         )
148     """
149
150     def _migrate_to_v1(self, cursor):
151         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
152         # Common fields are columns; detector-specific metrics go in `stats` JSON,
153         # so a new segmenter reuses this table by setting its own `detector`/`stats`.
154         cursor.execute("""
155             CREATE TABLE IF NOT EXISTS candidate_segments (
156                 id INTEGER PRIMARY KEY AUTOINCREMENT,
157                 video_id TEXT NOT NULL,
158                 detector TEXT NOT NULL,
159                 chunk_index INTEGER,
160                 start_time REAL NOT NULL,
161                 end_time REAL NOT NULL,
162                 duration REAL NOT NULL,
163                 confidence REAL,
164                 stats TEXT,
165                 detection_params TEXT,
166                 confirmed_segment_id INTEGER,
167                 human_label TEXT,
168                 notes TEXT,
169                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
170                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
171                 FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
172             DELETE SET NULL
173             )
174         """)
175         cursor.execute("""
176             CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
177             ON candidate_segments(video_id, detector)
178         """)
179         cursor.execute("PRAGMA user_version = 1")
180
181     def _migrate_to_v2(self, cursor):
182         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
183         # /api/process request. `status` is the EXECUTION state of the job
184         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
185         # that failed validation) lives inside `result` JSON as result["status"].
186         # `result` is the full sync-era response dict (incl. report paths), so the
187         # observability report is the job's artifact, per the plan.
188         cursor.execute("""
189             CREATE TABLE IF NOT EXISTS jobs (
190                 id TEXT PRIMARY KEY,
191                 video_path TEXT NOT NULL,
192                 video_id TEXT,
193                 segmenter TEXT,
194                 player_pool TEXT,
195                 max_frames INTEGER,
196                 status TEXT NOT NULL DEFAULT 'queued',
197                 progress TEXT,
198                 error TEXT,
199                 result TEXT,
200                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
201                 started_at TIMESTAMP,
202                 finished_at TIMESTAMP

```

```

203         """)
204         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
205         cursor.execute("PRAGMA user_version = 2")
206
207     def _migrate_to_v3(self, cursor):
208         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
209         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
210         # due
211         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
212         # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
213         # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
214         # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
215         # them.
216         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
217         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
218         TIMESTAMP")
219         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
220         next_poll_at)")
221         cursor.execute("PRAGMA user_version = 3")
222
223     def _migrate_to_v4(self, cursor):
224         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
225         # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
226         # path can scope rows to an owner. **NULL = local install = byte-identical to
227         # today** ? every existing row stays NULL and every existing query (which never
228         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
229         # the ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
230         # them.
231         # Additive ONLY: no served-behavior change rides on this slice.
232         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
233         TEXT")
234         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
235         user_id TEXT")
236         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
237         # of index churn while making the eventual per-user lookups cheap.
238         cursor.execute(
239             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
240             "WHERE user_id IS NOT NULL")
241         cursor.execute(
242             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
243             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
244         cursor.execute("PRAGMA user_version = 4")
245
246     def _migrate_to_v5(self, cursor):
247         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
248         # version): the resolved per-user `fusion_config` overlay + an INLINE
249         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
250         # evidence
251         # and provenance that earned it. `is_active` pins the one row the serving bridge
252         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
253         # slice
254         # only persists/loads it ? NOTHING reads it on the served path yet.
255         #
256         # user_id          ? owner of the model (NULL allowed = the local install)
257         # version          ? monotonic per-user model version
258         # baseline_version ? the shared baseline this was fit against (upgrade
259         # switch)
260         # feature_schema_hash ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
261         # gate
262         # fusion_config    ? JSON per-user FusionConfig overlay (cue_flags /
263         # thresholds)
264         # classifier_json  ? JSON LogisticRegression.to_dict() (None = config-only
265         # model)
266         # promotion_evidence ? JSON per_user_gate PromotionDecision (why it was
267         # promoted)

```

```

254         # n_videos_at_fit      ? held-out-user corpus size when fit (min_n trust floor
input)
255         # is_active            ? the one resolved row for this user (partial-unique
below)
256         cursor.execute("""
257             CREATE TABLE IF NOT EXISTS user_models (
258                 id INTEGER PRIMARY KEY AUTOINCREMENT,
259                 user_id TEXT,
260                 version INTEGER NOT NULL DEFAULT 1,
261                 baseline_version TEXT,
262                 feature_schema_hash TEXT,
263                 fusion_config TEXT,
264                 classifier_json TEXT,
265                 promotion_evidence TEXT,
266                 n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
267                 is_active INTEGER NOT NULL DEFAULT 0,
268                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
269             )
270         """)
271         # One model version per user (NULLs compare distinct in SQLite, so the local
272         # install can still carry multiple versioned rows; the active-row guard below
273         # is what enforces a single served model).
274         cursor.execute("""
275             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
276                 ON user_models(user_id, version)
277         """)
278         # At most ONE active model per user: a partial-unique index over the active
rows.
279         # (SQLite treats NULL user_id rows as distinct, so the local install's single
280         # active row is still guarded ? there is one local user.)
281         cursor.execute("""
282             CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
283                 ON user_models(user_id) WHERE is_active = 1
284         """)
285         cursor.execute("PRAGMA user_version = 5")
286
287         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
288             """Register or update a video row WITHOUT touching its child segments.
289
290             Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
291             with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
292             That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
293             before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
294             row in place; clearing segments is now an explicit, deliberate operation
295             (clear_video_data / prune_segments).
296             """
297             return self._execute_write(
298                 "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
299                 "ON CONFLICT(id) DO UPDATE SET "
300                 "filepath=excluded.filepath, status=excluded.status, "
301                 "validation_results=excluded.validation_results",
302                 (video_id, filepath, status, json.dumps(validation_results)),
303                 "Failed to add video",
304             )
305
306         def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
307             """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
308
309             Captured before a (re)segmentation so the run's new rows (id > watermark) can be

```

```

310         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
311         """
312         with self._connect() as conn:
313             cur = conn.cursor()
314             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
315                             (video_id,)).fetchone()[0]
316             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
317                             (video_id,)).fetchone()[0]
318             return int(p), int(c)
319
320     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
321                       play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
322                       cand_after: Optional[int] = None) -> None:
323         """Delete play/candidate segments by id range (events cascade from
play_segments).
324
325         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
326         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
327         and keep the OLD ones when the run failed/produced nothing (`*_after`).
328         """
329         with self._connect() as conn:
330             cur = conn.cursor()
331             if play_upto is not None:
332                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
333             if play_after is not None:
334                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
335             if cand_upto is not None:
336                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
337             if cand_after is not None:
338                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
339             conn.commit()
340
341     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
342         try:
343             with self._connect() as conn:
344                 cursor = conn.cursor()
345                 cursor.execute(
346                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
347                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
348                 )
349                 conn.commit()
350                 return cursor.lastrowid
351         except Exception as e:
352             logger.error(f"Failed to add play segment: {e}")
353             return None
354
355     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
356         return self._execute_write(
357             "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
358             (segment_id, timestamp, event_type, player, json.dumps(details or {})),
359             "Failed to add event",

```

```

360         )
361
362     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
363                               end_time: float,
364                               chunk_index: Optional[int] = None, confidence:
365                               Optional[float] = None,
366                               stats: Optional[Dict[str, Any]] = None,
367                               detection_params: Optional[Dict[str, Any]] = None) ->
368                               Optional[int]:
369         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
370         detector-specific dicts stored as JSON. Returns the new candidate id, or
371         None."""
372         try:
373             with self._connect() as conn:
374                 cursor = conn.cursor()
375                 cursor.execute(
376                     """INSERT INTO candidate_segments
377                     (video_id, detector, chunk_index, start_time, end_time, duration,
378                     confidence, stats, detection_params)
379                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
380                     (video_id, detector, chunk_index, start_time, end_time, end_time -
381                     start_time,
382                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
383                     {})))
384                 )
385                 conn.commit()
386                 return cursor.lastrowid
387         except Exception as e:
388             logger.error(f"Failed to add candidate segment: {e}")
389             return None
390
391     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
392         """Record that a candidate detection was confirmed as a given play_segment."""
393         return self._execute_write(
394             "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
395             (segment_id, candidate_id),
396             f"Failed to link candidate {candidate_id} to segment {segment_id}",
397         )
398
399     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
400                                 only_unlabeled: bool = False) -> List[Dict[str, Any]]:
401         """Fetch candidate detections for the annotation / fine-tuning workflow.
402         `stats` and `detection_params` are deserialized from JSON."""
403         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
404         params: List[Any] = [video_id]
405
406         if detector:
407             query += " AND detector = ?"
408             params.append(detector)
409         if only_unlabeled:
410             query += " AND human_label IS NULL"
411
412         query += " ORDER BY start_time"
413
414         candidates = []
415         with self._connect() as conn:
416             rows = conn.cursor().execute(query, params).fetchall()
417             for row in rows:
418                 d = dict(row)
419                 d["stats"] = json.loads(d["stats"] or "{}")
420                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
421                 candidates.append(d)
422         return candidates
423
424     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:

```



```

419         with self._connect() as conn:
420             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
421             if row:
422                 d = dict(row)
423                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
424                 return d
425             return None
426
427     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
428         """
429         Dynamically queries play segments based on filters:
430         - duration_min: float
431         - server: string
432         - receiver: string
433         - event_type: string (e.g. smash, foul)
434         """
435         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
436         params = [video_id]
437
438         if filters.get("duration_min"):
439             query += " AND r.duration >= ?"
440             params.append(filters["duration_min"])
441
442         if filters.get("server"):
443             query += " AND r.server = ?"
444             params.append(filters["server"])
445
446         if filters.get("receiver"):
447             query += " AND r.receiver = ?"
448             params.append(filters["receiver"])
449
450         if filters.get("event_type"):
451             # Check if there is an event of a specific type inside this segment
452             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
453             params.append(filters["event_type"])
454
455         segments_list = []
456         with self._connect() as conn:
457             cursor = conn.cursor()
458             rows = cursor.execute(query, params).fetchall()
459
460             for row in rows:
461                 r_dict = dict(row)
462                 r_id = r_dict["id"]
463                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
464
465                 # Fetch associated events
466                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
467                 r_dict["events"] = []
468                 for er in event_rows:
469                     e_dict = dict(er)
470                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
471                     r_dict["events"].append(e_dict)
472
473                 segments_list.append(r_dict)
474
475         return segments_list
476
477     def clear_video_data(self, video_id: str):
478         with self._connect() as conn:
479             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))

```

```

480         conn.commit()
481
482     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
483     # Writers follow the repo convention (swallow + logger.error, return bool); the
484     # worker logs loudly on a failed transition so a job can't silently wedge.
485
486     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
487                   player_pool: Optional[List[str]] = None,
488                   max_frames: Optional[int] = None,
489                   policy: Optional[Dict[str, Any]] = None) -> bool:
490         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
491         ExecutionPolicy
492         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
493         return self._execute_write(
494             "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
495             status, policy) "
496             "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
497             (job_id, video_path, segmenter,
498             json.dumps(player_pool) if player_pool is not None else None,
499             max_frames, json.dumps(policy) if policy is not None else None),
500             f"Failed to create job {job_id}",
501         )
502
503     def mark_job_running(self, job_id: str) -> bool:
504         return self._execute_write(
505             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
506             "progress='starting' WHERE id = ?", (job_id,),
507             f"Failed to mark job {job_id} running",
508         )
509
510     def set_job_progress(self, job_id: str, progress: str) -> bool:
511         return self._execute_write(
512             "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
513             f"Failed to set progress for job {job_id}",
514         )
515
516     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
517         return self._execute_write(
518             "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
519             f"Failed to set video_id for job {job_id}",
520         )
521
522     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
523         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
524         validation) is inside `result` ? job status 'done' means 'the worker
525         finished'."""
526         return self._execute_write(
527             "UPDATE jobs SET status='done', progress='finished', result=?, "
528             "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
529             (json.dumps(result), job_id),
530             f"Failed to mark job {job_id} done",
531         )
532
533     def mark_job_failed(self, job_id: str, error: str) -> bool:
534         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
535         return self._execute_write(
536             "UPDATE jobs SET status='failed', error=?, "
537             "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
538             f"Failed to mark job {job_id} failed",
539         )
540
541     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
542         """Fetch one job; `result`/`player_pool` JSON deserialized."""
543         with self._connect() as conn:
544             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",

```

```

(job_id,)).fetchone()
542         if not row:
543             return None
544         d = dict(row)
545         d["result"] = json.loads(d["result"]) if d["result"] else None
546         d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
547         d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
548         return d
549
550     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
-----
551     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
552         """Attach/replace a job's JSON ExecutionPolicy."""
553         return self._execute_write(
554             "UPDATE jobs SET policy=? WHERE id = ?",
555             (json.dumps(policy) if policy is not None else None, job_id),
556             f"Failed to set policy for job {job_id}",
557         )
558
559     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
560         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
561         worker. ``claim_due_job`` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
562         computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
563         return self._execute_write(
564             "UPDATE jobs SET status='waiting', "
565             "next_poll_at=datetime('now', ?), "
566             "progress=COALESCE(?, progress) WHERE id = ?",
567             (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
568             f"Failed to set job {job_id} waiting",
569         )
570
571     def seconds_until_next_due(self) -> Optional[float]:
572         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
573         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
574         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
575         try:
576             with self._connect() as conn:
577                 row = conn.cursor().execute(
578                     "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
579                     "    (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
580                     "FROM jobs WHERE status='waiting']").fetchone()
581                 if row is None or row["secs"] is None:
582                     return None
583                 return max(0.0, float(row["secs"]))
584         except Exception as e:
585             logger.error(f"Failed to compute next-due delay: {e}")
586             return None
587
588     def claim_due_job(self) -> Optional[Dict[str, Any]]:
589         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
590         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
591         double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
592         first. Returns the claimed job dict, or None if nothing is runnable."""

```

```

593         try:
594             with self._connect() as conn:
595                 cur = conn.cursor()
596                 row = cur.execute(
597                     "SELECT id, status FROM jobs WHERE status='queued' "
598                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
599                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
600                 if not row:
601                     return None
602                 cur.execute(
603                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
604                     "WHERE id=? AND status=?", (row["id"], row["status"]))
605                 conn.commit()
606                 if cur.rowcount != 1:
607                     return None # lost the race to a concurrent
worker
608                 return self.get_job(row["id"])
609         except Exception as e:
610             logger.error(f"Failed to claim a due job: {e}")
611             return None
612
613     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
614         """Crash recovery: any job still queued/running from a previous process is dead
615         (the executor is in-process). Mark failed so clients see a terminal state, not a
616         hang. Called once at startup. Returns the number of jobs swept."""
617         try:
618             with self._connect() as conn:
619                 cur = conn.cursor()
620                 cur.execute(
621                     "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
622                     "WHERE status IN ('queued', 'running')", (reason,))
623                 conn.commit()
624                 return cur.rowcount
625         except Exception as e:
626             logger.error(f"Failed to sweep stale jobs: {e}")
627             return 0
628
629     # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
630     # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
631     # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
632     # serving.py) resolves them, and wiring that bridge into the production decision
seam
633     # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
634     # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
635     # JSON-serialised dicts; the rest are scalar columns.
636
637     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
638                           baseline_version: Optional[str] = None,
639                           feature_schema_hash: Optional[str] = None,
640                           fusion_config: Optional[Dict[str, Any]] = None,
641                           classifier_json: Optional[Dict[str, Any]] = None,
642                           promotion_evidence: Optional[Dict[str, Any]] = None,
643                           n_videos_at_fit: int = 0,
644                           is_active: bool = False) -> Optional[int]:
645         """Insert or replace ONE per-user model row keyed by ``(user_id, version)``.
646
647         Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
648         the
649         rowid). When ``is_active`` is True this row becomes the user's single active
650         model ?
651
652         any previously-active row for the SAME user is first cleared, so the
653         ``idx_user_models_one_active`` partial-unique invariant always holds (one active

```

```

model
651         per user). Returns the row id, or None on failure.
652         """
653         try:
654             with self._connect() as conn:
655                 cur = conn.cursor()
656                 if is_active:
657                     # Demote the current active row first (NULL user_id = local: IS NULL
match).
658                     if user_id is None:
659                         cur.execute(
660                             "UPDATE user_models SET is_active = 0 "
661                             "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
662                             (version,))
663                     else:
664                         cur.execute(
665                             "UPDATE user_models SET is_active = 0 "
666                             "WHERE user_id = ? AND is_active = 1 AND version != ?",
667                             (user_id, version))
668                 cur.execute(
669                     """INSERT INTO user_models
670                     (user_id, version, baseline_version, feature_schema_hash,
671                     fusion_config, classifier_json, promotion_evidence,
672                     n_videos_at_fit, is_active)
673                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
674                     ON CONFLICT(user_id, version) DO UPDATE SET
675                     baseline_version=excluded.baseline_version,
676                     feature_schema_hash=excluded.feature_schema_hash,
677                     fusion_config=excluded.fusion_config,
678                     classifier_json=excluded.classifier_json,
679                     promotion_evidence=excluded.promotion_evidence,
680                     n_videos_at_fit=excluded.n_videos_at_fit,
681                     is_active=excluded.is_active""",
682                     (user_id, version, baseline_version, feature_schema_hash,
683                     json.dumps(fusion_config) if fusion_config is not None else None,
684                     json.dumps(classifier_json) if classifier_json is not None else
None,
685                     json.dumps(promotion_evidence) if promotion_evidence is not None
else None,
686                     n_videos_at_fit, 1 if is_active else 0))
687                 conn.commit()
688                 rid = cur.execute(
689                     "SELECT id FROM user_models WHERE "
690                     + ("user_id IS NULL" if user_id is None else "user_id = ?")
691                     + " AND version = ?",
692                     ((version,) if user_id is None else (user_id, version))).fetchone()
693                 return int(rid[0]) if rid else None
694             except Exception as e:
695                 logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
{e}")
696                 return None
697
698     def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
699         """Return the single active per-user model row for ``user_id`` (NULL = local
install),
700         with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
701         or None if the user has no active model. Does not touch the served pipeline."""
702         with self._connect() as conn:
703             if user_id is None:
704                 row = conn.cursor().execute(
705                     "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
706             else:
707                 row = conn.cursor().execute(

```

```

708             "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
709             (user_id,)).fetchone()
710         if not row:
711             return None
712         d = dict(row)
713         for col in ("fusion_config", "classifier_json", "promotion_evidence"):
714             d[col] = json.loads(d[col]) if d[col] else None
715         return d
716

```

5. user (2026-06-20T08:00:33.199Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-209 of 384 total (38930 tokens, cap 25000). Call Read with offset=210 limit=209 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```

1      # CODE_MAP.md
2
3      **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-
4      hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
5      segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
6
7      **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this
8      doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure
9      (this revision follows #40).
10
11      ? **2026-06-10 audit-hardening sweep changed several behaviors documented below ? trust
12      git history over stale lines until the next full regen:** `add_video` is now an **UPSERT** (no
13      FK-cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via
14      `segment_watermarks`/`prune_segments`); detector artifacts are keyed by **video_id**, not
15      basename; **`indexing.wasb.*` is now a real typed config** (was silently dropped); `timeout_sec`
16      defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**;
```

`match\_intervals` requires real overlap (iou>0); `run\_regression` baseline lives in tracked
`eval\_baselines/` and **exits 3 on a missing baseline**;

the API validates
`video\_path`/`output\_filename`. New tests: `test\_api\_endpoints`, `test\_compiler`,
`test\_wasb\_runner`, `test\_reingest\_safety`, `test\_detector\_keying`, `test\_config\_wasb`,
`test\_path\_safety`, `test\_reliability\_hardening` (+ collector `test\_licensing\_gate`,
`test\_import\_guard`, `test\_cvat\_robustness`).

```

8
9      **LEGEND**
10      - `@path` = file (always repo-relative)
11      - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
12      - `->` = dataflow / call / "produces"
13      - `$` = data contract (canonical shape)
14      - `?` = gotcha / footgun
15      - `?` = registry / plugin extension point
16      - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
17
18      ---
19
20      ## 0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)
21
22      Top-level tree and the **placement rule** for each kind of file. When unsure, prefer an
23      existing
24      **registry / extension point** (`?`) over a new top-level file.
25
26      | You're adding? | Put it in? | Notes |
27      |---|---|---|
28      | Backend / runtime code | `backend/<subsystem>/` |
29      `pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`,
30      `infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most
31      additions are **register a class in the relevant `?` registry**, not a new module tree. |
32      | **Eval LIBRARY** (importable, pure ? metrics, calibration, a scorer, a feature) |
33      `backend/eval/` | Pure core, no side-effects at import; CLI/I-O lives in a separate driver.

```

```

These are **imported widely** (e.g. `calibrate_wasb` by 8 sites) ? they are library, not
scripts; do NOT move them out. |
29 | **Standalone run-driver / one-off entry CLI** (has `__main__`, NOT imported by app
code) | **scripts/** | e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`.
Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing
backend, add `scripts/<name>.py" = ["E402"]` to `ruff.toml`. |
30 | **Ops / maintenance tool** (not part of the served app ? cache GC, batch admin) |
`tools/` | Run as `python -m tools.<name>` (it's an importable package). e.g.
`tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested. |
31 | Training code (model fitting) | `training/` | Behind the CI-enforced import wall
(ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`. |
32 | A test | `tests/test_*.py` | pytest auto-discovers; the full-suite lane has a **70%
coverage floor**. Pure/offline + mocked (no GPU/WSL/network). |
33 | A doc (design, plan, living log) | `docs/` | Index it in `docs/README.md`. Only
**terminal-state** (DONE/REJECTED) work moves to `docs/archives/`. |
34 | A throwaway repro / debug script | `scratch/` | **Gitignored** + excluded from
ruff/pytest. Never imported by app code. |
35 | A model artifact (weights + manifest) | `artifacts/<name>/<ver>/` | ADR-008:
**manifest committed** (provenance), **weights gitignored**. |
36 | Eval baseline / leaderboard JSON | `eval_baselines/` | Tracked (survives a clean
checkout); the no-regression gate reads it. |
37 | **Committed regression fixture** (video-free golden) |
`eval_baselines/fixtures/<slug>/` | Tracked, **LF-pinned** (`gitattributes`). Synthetic Tier-0
(or owner-cleared, de-attributed real). Drives the `golden_fixtures` characterization gate with
**no video/GPU/DB**. See `docs/GOLDEN_REGRESSION_FIXTURES.md`; mint via `python -m
backend.eval.golden_fixtures --freeze-synthetic` / `--freeze-real` (refusal-gated). |
38 | Pipeline run outputs (reels, trajectories, DBs, frames) | `output/` | **Gitignored.**
Per-video isolation is planned in `PER_VIDEO_OUTPUT_LAYOUT.md`. |
39
40 **Canonical files that stay at the repo ROOT** (referenced by CI / packaging / docs ? do
not move):
41 `pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) .
`run_all_tests.py` (CI: `ci.yml`) .
42 `run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) .
`run_eval.py`
43 (grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic
`setup.py` moved to
44 `scripts/doctor.py` (it shadowed the build system); invoke via `python
scripts/doctor.py` or `indexer --setup`.
45 New standalone run-drivers go in **scripts/**, not the root.
46
47 ---
48
49 ## 1. PIPELINE
50
51 ### 1A. Runtime: video file -> highlight reel
52 ```
53 typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
54 @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
55 -> video_id = MD5(whole file) @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
56 -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
57 FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
58 [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
59 -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
60 -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')

```

```

61     -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
62     -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
63     [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
64     -> finally: emit_indexer_report(...) @backend/reporting/__init__.py (? ALWAYS runs
? even on validation failure / exception; NEVER raises; grafts response["reports"])
65     -> select segment ids -> [(start,end)] (+ stitching padding)
66     -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
67     per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
68     ```
69     Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
70     ```
71     P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
72     -> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted
via db.add_candidate_segment]
73     -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt) ? provider_registry
74     -> P4 db.add_play_segment + db.link_candidate_to_segment
75     ```
76     Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
77     Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).
78
79     ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
80     ```
81     WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
82     -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
83     -> WasbRunner.run_predict(video, out_dir)
84     stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
85     -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
86     -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
87     -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING
BRAIN]
88     ```
89     (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
90
91     ### 1C. Calibration / eval flow (NO pipeline run unless told)
92     ```
93     GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
94     DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval] stage
? {candidates, final}
95     -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
96     -> harness.evaluate -> EvalReport -> report_to_dict
97     -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
98     -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py
[CI gate]
99     Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
100     Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned
classifier) @backend/eval/{calibrate_local,distill_local}.py
101     Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save

```



```

102     Gemini refinement driver (tuning, standalone):
gemini_refine.{refine_window,full_video_rallies} @backend/eval/gemini_refine.py
103     Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
104     ...
105     Entrypoints: `@run_eval.py` (single video score), `@scripts/run_phase3_eval.py`
(manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@scripts/nightly_regression.py` (nightly golden-corpus rally-quality tripwire, exit
1=regression/4=stale), `@backend/eval/calibrate_wasb.py` +
`@backend/eval/export_wasb_segments.py` (WASB tuning drivers).
106
107     ---
108
109     ## 2. SUBSYSTEMS
110
111     ### 2.0 Orchestration & config
112     - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS).
Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig`
(INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces
stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI
summaries may still use `print` for direct output.
113     - `::app` FastAPI; `::db_instance`, `::global_config` (`Optional[AppConfig]`) module
globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` ->
every endpoint NPES.
114     - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_drain_due_jobs)` -> **202
+ job_id*. `::drain_due_jobs` / `::run_claimed_job` (worker loop EXTRACTED to
`@backend/api/job_worker.py` (#234), re-exported on `backend.main`): claim each runnable job via
`db.claim_due_job` -> `_execute_processing` -> mark done/failed (a `JobWaiting` parks the job
via `set_job_waiting` for resume; EXECUTION_POLICY P3 self-scheduling). `::execute_processing`
= the former sync body (hash -> validate -> add_video -> segment -> `finally:` always
`emit_indexer_report`, grafts `response["reports"]`; never raises from the report); a RAISING
segmenter now prune_after-restores pre-run rows before re-raising. `::get_job` `@GET
/api/jobs/{job_id}` -> `{status, progress, video_id, error, result, reports, ...}` ? job
`status` = EXECUTION state (`queued|running|done|failed`); the pipeline verdict lives in
`result["status"]`. `::get_executor` lazy module-global `ThreadPoolExecutor(max_workers=1)`
(single box + Gemini serial-upload learning; tests monkeypatch it inline). `main()` runs
`db.fail_stale_jobs()` at startup (orphaned queued/running -> failed). `::compile_highlights`
`@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET
/api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs
`indexing.motion_threshold` (dual model/dict read). ? CLI `--video-path` mode still runs the
pipeline synchronously inline (separate code path, unchanged by #16).
115     - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in
`@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`;
`uploads.py::upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the
startup config, and a lazy `import backend.main` avoids an import cycle) ?
`uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext
allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-
bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-
part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST ../complete`
(`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ?
client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ?
upload state (`uploads.py::uploads` + `::uploads_lock`) is IN-PROCESS ? restart aborts
partials (client re-inits); per-user partitioning lands with PR-3. `main.py::download_highlight`
`@GET /api/highlights/{video_id}/{filename}` (download was NOT moved ? still in main.py) ? safe-
name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert
path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads',
max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge
proxies cap bodies ~100MB).
116     - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`) ; kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
117     - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.

```

```

`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
118     - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default `segmenter` fallback `motion`.
119     - `@scripts/run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive
`input()` (blocks automation), skips validation. Config via `backend.config.load_config`
(typed); segmenter, padding, and db path all read from the model (no literal fallbacks).
120     - `@run_eval.py::main` ? score one video's DB preds vs GT CSV.
`--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed.
`::default_db_path` resolves via `backend.config.load_config`
(`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
121     - `@scripts/run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score,
aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as
label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed
`AppConfig` (same object the live pipeline feeds segmenters).
122     - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic:
**0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare
(snapshots even a regressing run); `::default_db_path` resolves via
`backend.config.load_config`.
123     - `@scripts/nightly_regression.py::main` ? **nightly rally-quality tripwire** (the dual-
eval-set discipline made automatic). Re-scores the golden corpus
(`output/human_lovo_manifest.json`) under TWO configs ? `default` (SERVED, milestone knobs OFF)
+ `milestone` (R3 cap=6 + R4 inactivity trim) ? via `backend.eval.rally_seg_eval.evaluate` (CPU,
seconds; CACHED trajectories, **no GPU/Gemini/WASB**), diffs headline metrics (`tol_f1`, tIoU
`f1@0.5/0.25`, `merge_rate`) + the **per-video over-seg guard** (`split_count`/`merge_count` may
not rise on ANY video) vs frozen `eval_baselines/nightly_baseline.json`. Pure core
(`summarize_run`/`compare_config`/`compare_all`/`format_report`) is unit-tested; thin I/O shell
scores + writes `output/nightly_regression/<date>.{md,json}`. Exit **0=pass, 1=regression,
2=missing corpus, 3=no baseline, 4=stale (corpus/config changed ? re-snapshot)**. `--update-
baseline` snapshots HEAD. **Default-ADDITIVE ? does NOT change the promotion gate.** Wired as a
local Windows Scheduled Task via `scripts/nightly_regression.ps1` (wrapper) +
`scripts/register_nightly_task.ps1` (a cloud agent can't ? the cached trajectories live outside
the repo). The future **RAW eval set** (currently-bad videos) plugs in as a third config. See
`docs/R2_EVAL_METRIC_DESIGN.md` + `memory/eval-dual-set-regression`.
124
125     - `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports
`load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`,
`StitchingConfig`, `ValidationConfig` (from `.models`); `__all__` = exactly those 7. Use `from
backend.config import load_config, AppConfig`.
126     - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for
defaults**.
127     - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`,
`ai_model='gemini-2.5-flash'`; sections `validation`, `indexing`, `output`, `stitching`
(`default_factory`). `model_config` = {extra:'allow', validate_assignment:True}. ?
`extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
128     - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain
dicts** (`model_dump(mode='json')`, NOT the model) so chained
`config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any
section (sections discovered dynamically from `model_fields`). ? "bit every validator on real
runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises
`KeyError` for unknown top-level keys else delegates to `.get`.
129     - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`,
`min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`,
`chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`,
`ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested
`tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `gemini`. ?
`::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ?
a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`).
130     - `::ValidationConfig` (`min_resolution_width=1280`, `min_resolution_height=720`, `min_f
ps=29.9`, `min_blur_threshold=20.0`, `max_scene_cuts_per_minute=0.5`, nested `relevance`);
`::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`).
`::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`,
`conda_env='TrackNetV4'`, `weights_path='', `queue_length=5`, `velocity_threshold=0.02`).
`::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`,

```

```

`output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`).
`::StitchingConfig`
(`begin_padding=0.5`,`end_padding=0.5`,`use_cache=False`,`min_shot_count=1`). `::Sport` Literal
of 5 sports. `::Config = AppConfig` alias.
131 - `@backend/config/loader.py` ? load/save with defaults-merge.
132 - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT repo-root).
`::get_built_in_defaults` = `AppConfig().model_dump(...)`
133 - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-
level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole
defaults dict for that section (Pydantic then re-applies field defaults for missing leaves).
Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-
fatal).
134 - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present
(used by scripts/doctor.py/--setup). `::get_default_config` ? ? transitional plain-dict alias.
135 - `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**indexing.default_segmenter='gemini'** diverges from model default `yolo_hybrid` ? file
wins at runtime.
136 - `@scripts/doctor.py` (formerly root `setup.py`) ? `::init_default_config` ? delegates
to `backend.config.save_default_config` (single-source defaults; skips if config.json exists).
`::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python>3.8,
ffmpeg/ffprobe on PATH, optional torch+CUDA). Run via `python scripts/doctor.py` or `indexer
--setup`.
137
138 ### 2.1 Segmenters (under pipeline/segmenters) ?
139 - `@backend/pipeline/segmenters/base.py`
140 - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`,`description`,`process_video`. $ `process_video(video_id, video_path, player_pool=None,
max_frames=None)` -> List[dict{id,start_time,end_time,duration,metadata}]`.
141 - `::SegmenterRegistry`,`::segmenter_registry` (singleton). ? **register a new
segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module
bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an
IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` ->
name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-
instantiates + swallows exceptions into "No description available." (a throwing `description` is
masked).
142 - `@backend/pipeline/segmenters/__init__.py` ? imports all 6
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid,fusion_hybrid`) = the de-facto
registration manifest; re-exports them + `segmenter_registry` via `__all__`.
143 - `@backend/pipeline/segmenters/trajectory_hybrid.py::TrajectoryHybridSegmenter` ?
SHARED ENGINE for the trajectory hybrids (2026-06-11 collapse of the copy-paste twins; the "fix
BOTH" footgun is retired): healthcheck gate -> P2 trajectory->windows (+persist) -> P3 provider
handoff -> P4 DB/link; single `::shot_count_from` (old wasb/tracknet shot_count drift gone).
NOT registered itself. ? Subclass contract: `family` (config section
`indexing.<family>.velocity_threshold(0.02)`,`output/<family>` trajectory dir,
`<family>-hybrid-<model>` metadata tag), `display_label` (log lines), `name`/`description`
properties, `_build_runner(idx_cfg)` -> (DetectorRunner, detection_params extras). **Two fusion
seams (IDENTITY by default ? twins byte-identical, pinned by the equivalence test):
`::enrich_candidate_stats(cw,points,fps,frame_width,video_path,idx_cfg)`** (base = the 4
motion-key stats dict; computed once into `window_stats`, used for persist + the handoff gate)
**and `::select_for_handoff(windows>window_stats,idx_cfg)`** (base = all indices). ? shared
knobs still YOLO-named (`log_yolo_candidates/store_yolo_candidates`); whole-video single pass;
needs `{PROVIDER}_API_KEY` (missing -> []). Equivalence pinned by
`tests/test_trajectory_hybrid_equivalence.py` (detector identity, shot_count, offsets, + the
base-hook-identity tests ? parametrized over both twins).
144 - `@backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (name
`fusion_hybrid`, **default-OFF** multi-signal fusion P2) ? subclasses the engine;
`family='fusion'`,`display_label='Fusion'`. `::build_runner` selects the substrate via
`indexing.fusion.substrate` ('wasb'|'tracknet') reusing the existing runner+config.
`::enrich_candidate_stats` overrides to ALWAYS add `net_crossings` (Gate A signal, GPU-free via
`rally_gate.gate_windows(..., estimate=self._axis_estimate(points))` ? axis/net cached per
trajectory) and, only when `fusion.cue_flags.person`, the ~5 person features
(`::person_features` ? lazy `PersonDetector.detections_per_frame` over the window's ORIGINAL-
video frame range ? `fusion_features.compute_person_features`). `::select_for_handoff`
overrides to apply served Gate A (`fusion.min_crossings`) ? Gate B (`fusion.min_players`,
person-only) ? default 0/0 ? all windows kept. ? persisted candidates are ALWAYS the full

```

```

superset (offline substrate intact); only the handoff/play_segments are gated. ? to A/B vs a
tuned wasb run, set `fusion.velocity_threshold = wasb.velocity_threshold` (the engine reads
`idx_cfg['fusion'].velocity_threshold`). ? person features eager even for gated windows (P3
cleanup). Tests: `tests/test_fusion_hybrid.py`.
145 - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`'wasb_hybrid'`, the LIVE shuttle substrate; MIT, ships pretrained badminton weights) ? thin
subclass: `family='wasb'`, wires `WasbRunner`/`WasbConfig`; `detection_params` extras =
`weights_path` only.
146 - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`'tracknet_hybrid'`) ? thin subclass: `family='tracknet'`, wires
`TrackNetRunner`/`TrackNetConfig`; extras = `weights_path` + `queue_length`.
147 - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name
`'yolo_hybrid'`, no YOLO remains; ADR-005). **Stage-1 detection EXTRACTED to
`detectors/person_detector.py` (fusion P1, no behavior change)** ? the segmenter now
ORCHESTRATES only (chunking, pipelined-vs-sequential P2 prefetch where ffmpeg overlaps
sequential GPU, AI-provider handoff, candidate persist/link) and delegates detection to
`self.person` (`PersonDetector`). Imports `_DEFAULT_DETECTOR` from person_detector for the
`detection_params` snapshot. ? `yolo_tracker`/`bytetrack.yaml` is dead/back-compat; `gemini_*`
config keys are deprecated aliases still honored; still mpy-quarantined for pre-existing
orchestration debt (the torchvision code that caused it left).
148 - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ?
pure-AI, no P2. Chunks are RE-ENCODED + downsampled to `indexing.ai_chunk_scale_width` (default
1280; 0 = legacy stream-copy) before upload ? stream-copied 4K chunks exceed the provider's
`MAX_UPLOAD_MB` and every chunk gets refused ("0 segments / success"); re-encode also fixes
keyframe drift at chunk boundaries. Oversize refusals (provider metrics `oversize_skipped`) are
aggregated -> `logger.error` + `reporting.run_signals` (`oversize_clips_skipped`) for the report
rule. Workers from `indexing.ai_max_workers`. ? chunks only if `duration>chunk_duration`;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
149 - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ?
pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
150
151 ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
152 - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() ->
Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately
matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring
includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on
threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL
de-risk seam (low-regret 6 / review item 1).
153 - `@backend/pipeline/detectors/tracknet_runner.py`
154 - `::TrackNetConfig` (+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro`
field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
155 - `::to_wsl_mnt_path` (`C:\x`->`\mnt/c/x`; ? POSIX/~` passthrough, UNC unhandled),
`::TrajectoryPoint` (frame,visible,x,y).
156 - `*:::parse_trajectory_csv` @staticmethod** + `*:::trajectory_to_action_windows`
@staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path`
defaults `''` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names
`<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`,
`frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout
(hangs forever). `::stage_video` "OK" sentinel.
157 - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner)
(+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default
`wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::INFER_WIN`) +
video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` =
`staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-
named `<stem>_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until
staged); `wsl_bash` now shared via `detectors/base.py::WslCommandMixin` (2026-06-11 dedupe).
Both runners now satisfy the DetectorRunner ABC for future swappability. **Cache hygiene
(#109):** on a SUCCESSFUL run both runners delete their WSL intermediates ? WASB the frame cache
+ staged video, TrackNet the staged video ? gated by `indexing.{wasb,tracknet}.keep_frames`
(default false; on FAILURE kept so a re-run resumes). Pre-run `min_free_gb` disk-guard warning.
Shared helpers `WslCommandMixin::wsl_rm_rf`/`wsl_free_gb`.

```

```

158     - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT
repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache:
`::DET_FILE='det_raw.jsonl'`, `::MANIFEST_FILE='manifest.json'`, `::CACHE_VERSION=1`,
`::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to
heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights`
compared raw), `::atomic_write_text/atomic_write_trajectory`, `::dets_to_tracker` (xy MUST be
np.array), `::build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW),
`::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker
STATEFUL -> always fully re-run; `--fresh` ignores cache.
159     - `@backend/pipeline/detectors/person_detector.py::PersonDetector` ? the **PERSON rally
cue** (torchvision detector + supervision ByteTrack + velocity windowing), extracted from
yolo_hybrid (fusion P1; type-clean, NOT quarantined). `::PERSON_CLASS_ID=1` ? (ultralytics used
0; wrong value -> zero detections),
`::DETECTOR_FACTORIES`/`::DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. `::lazy_load_model`
(lazy `torchvision` import so module/tests load without it; `self.model` typed `Any`),
`::make_tracker` (per-chunk ByteTrack ? IDs chunk-local), `::detect_and_track` (one frame ->
confident persons w/ track IDs; BGR->RGB, COCO person filter, ByteTrack association),
`::extract_action_windows_from_chunk` (chunk -> high-motion windows w/ `peak/mean_velocity`,
`active_frame_count`, `track_id_count` ? the stats that feed candidate `stats` AND the future
learned-fusion person features). ? velocity at `effective_fps = fps/frame_skip` ->
`yolo_velocity_threshold` coupled to `frame_skip`. Consumed by yolo_hybrid AND the fusion
segmenter (P2 reuses the SAME producer over shuttle-seeded windows via `::detections_per_frame`
? add-a-producer, not a refactor). BSD torchvision + MIT ByteTrack.
**`::detections_per_frame(video_path,start_frame,end_frame,frame_skip)`** (fusion P2) reads the
ORIGINAL video by ABSOLUTE frame range (seek+sequential) ? `{frame_idx: [(track_id,bbox)]}` +
`fps/frame_width/frame_height`, so indices align 1:1 with the WASB trajectory `.frame` (no clip
re-timing). ? cv2 frame-seek accuracy varies by codec ( $\pm$  a few frames on real footage).
160     - `@backend/pipeline/detectors/fusion_features.py` ? **pure** per-window person-cue
feature math (no I/O/torch/cv2; mirrors rally_gate discipline). Inputs: per-frame
`{frame_idx: [(tid,bbox)]}` + the in-window shuttle points + optional normalized `court_polygon`
+ `net=(axis,pos)`. `::point_in_polygon` (ray-cast), `::person_in_court` (foot-point; None
polygon=count-all; bad dims=False), `::in_court_counts`, `::players_in_court` (median),
`::in_court_ratio`, `::mean_player_motion` (per-track centre displacement; x by width, y by
height), `::two_sided_motion` (both net-halves occupied), `::shuttle_player_colocation` (shuttle
inside a person box  $\pm$  `pad_frac` halo over aligned visible frames ? **the held-shuttle
discriminator**), `::compute_person_features` (aggregator ? the 5 `::PERSON_FEATURE_NAMES`). All
scale/translation-invariant, graceful to None/empty. Tests: `tests/test_fusion_features.py`.
161     - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure,
no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> 'x'),
`::count_net_crossings` (sign changes vs median net; deadband jitter zone),
`::gate_windows`/`::apply_gate` (`min_crossings=2` default; kills single-toss service feeds;
**`estimate` param** lets a caller scoring many windows pass a precomputed `(axis,net,span)` to
skip per-window re-estimation ? used by FusionSegmenter).
`::GatedWindow` (start,end,crossings,visible_points,kept). ? window contract here is
`(start,end)` tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local /
distill_local / export_wasb_segments / fusion_hybrid.
162
163     ### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
164     - `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`,
`::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict`) = wire shape),
`::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN)
for mean_iou/boundary errors.
165     - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end`
CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
166     - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video
k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video;
needs 22 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT),
`::select_best`, `::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. $
`PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap
>0.1` = overfit alarm.
167     - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` ($`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,

```

```

make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
168     - `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing +
net-crossing gate, no cloud at runtime). `::local_grid` (180 cfs), `::make_local_predict_fn`
(adds `rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a
labels file yields no rallies), `::run`/`::main`. $
`LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.
169     - `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison
+ optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video),
`::SEG_FIELDS`, `::COMP_FIELDS`, `::build_comparison`, `::seconds_to_mmss`, `::maybe_slice`
(lazy-imports rally_slicer). `--slice` requires `--video`. ? `write_segments_csv` output loads
straight into `rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is
load-bearing.
170     - `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in
run_phase3_eval). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1),
`::default_stages_for` (`auto`; `::FINAL_ONLY_SEGMENTERS={motion,gemini}`),
`::resolve_video_path` (normalizes collector `./data..` Windows paths), `::format_aggregate`.
171     - `@backend/eval/harness.py` ? DB-pred scorer (no re-run).
`::STAGES=('candidates','final')`, `::get_predictions`
(candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})`); ? unknown
stage raises ValueError; reused by regression.py),
`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate), `::format_report_text`.
172     - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES`
(5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default
0.0), `::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->(X,y,intervals); `feature_fn` is the substrate plug-in seam.
173     - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `sigmoid`
clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON `"model":"logreg"` tag.
174     - `@backend/eval/eval_partial_labels.py` ? score pipeline vs **PARTIAL human labels**
(PR #60). `::restrict_to_window` (drop preds outside `[0, last GT end]` so un-labeled-tail
detections aren't FPs ? THE partial-label methodology), `::run` (baseline/+gate/tuned/+gate ops
table + per-rally diff via `export_wasb_segments.build_comparison`), `::sweep`/`--sweep` (grid
vs labels; ? fit-on-self = optimistic), `::DEFAULT_GRID`, `::TUNED=(0.05,1.0,1.0)`. ?
`window_end` inferred from `max(GT end)` is WRONG if a video is labelled PAST its last rally ?
P1 fix = collector persists `labeled_window` in the manifest. Reuses calibrate_wasb + rally_gate
+ metrics.
175     - `@backend/features/rally_seq.py` ? **THE single source of truth for the Gen-0
rally_seq per-frame feature schema** (ADR-008): training (`training`) AND the serving/eval path
import the SAME `build_frame_arrays`/`features` so train==serve by construction. Pure
numpy/scipy (no torch/cv2). `::FEATURE_SCHEMA_VERSION` (=1; bump on ANY semantic change ?
artifacts pin it, loader hard-fails on mismatch), `::FEAT_NAMES` (the 6:
`vis_rate,spd_mean,spd_max,cross_rate,x_std,y_std`), `::FEATURE_SCHEMA`
(`win_sec`/`stride_sec`/`fps_semantics`/`occlusion_rule`),
`::build_frame_arrays(points,frame_width,fps)` (per-frame vis/x/y + per-visible-frame speed &
net-crossings; speed normalized per-SECOND `fps`, NOT per-frame ? the 2026-06-10 fps-skew fix;
skips speed/crossings across occlusion gaps > `max(2,round(0.25*fps))`),
`::features(arr,fps,win_sec,stride_sec)` (scipy `uniform/maximum_filter1d` windowed -> `(X[n,6],
times)`). ? re-exported by `@backend/eval/rally_seq_proto.py`.
176     - `@backend/eval/rally_seq_proto.py` ? **PROTOTYPE** learned per-FRAME rally segmenter =
owned-model **Gen-0 seed** (PR #61). 6 features over the WASB trajectory (`vis_rate, spd_mean,
spd_max, cross_rate, x_std, y_std`) -> `classifier.LogisticRegression`, honest LOVO. ?
`build_frame_arrays`/`features` (+`FEAT_NAMES`) are NOT defined here ? they are **re-exported**
(`noqa F401`) from `@backend/features/rally_seq.py` (ADR-008 single-sourcing; speed per-SECOND +
occlusion-gap-guarded since the 2026-06-10 sweep). Locally-owned: `::labels_for`,
`::probs_to_segments` (smooth->threshold->merge/min-dur), `::roc_auc` (rank-based with **midrank
ties**, scipy.stats.rankdata), `::run` (per-video AUC + LOVO-threshold F1 + oracle-threshold
F1). ? DIRECTIONAL only (n=6, linear, threshold-transfer unsolved); held-out per-frame AUC
0.83?0.92, **beats the heuristic on multi-court footage** (Kushagra/gBaaddy). **n=6 LOVO:
learned 0.626 vs heuristic 0.480** (learned rose from 0.583 after the fps-normalization fix).
Manifest `output/human_lovo_manifest.json` (n=6; trajectory paths now under `Annotation
Setup/Collect/Trajectories/`) shared w/ calibrate_local.

```

```

177 - ? **MULTI-COURT / background-game confound (2026-06-08 finding):** WASB tracks EVERY
shuttle in frame incl. other courts; videos with background games (Kushagra, gBaaddy) have high
dead-time shuttle visibility (41?57% vs 18?22% clean) ? the velocity-windowing heuristic can't
find clean rally boundaries (F1 0.16/0.28 vs ~0.75 on single-court). Diagnostic = in-rally÷dead-
time visibility "contrast" (?3.6× heuristic works, ?1.9× fails). Lever = **foreground-court
isolation** (spatial gate to the prominent court, needs real court-region detection). Academy =
multi-court = the target env. See `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
178 - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond
calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT:
duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal),
`::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`),
`::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-
one-video-out needs ?2 videos; final model trained on all.
179 - `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT
padded clips (precision lever; eval/tuning, NOT the live pipeline).
`::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (?
FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`,
`::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises
SystemExit); imports `BASELINE` from calibrate_wasb +
`TUNED`/`write_segments_csv`/`seconds_to_mms` from export_wasb_segments.
180 - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=eval_baselines/regression_baseline.json` (tracked, NOT under gitignored
output/), `::regress`, `::regress_with_provider` (GT via ?`annotation_registry.get(tier)`;
"file"->FileProvider/"vendor"->HumanVendor/"auto"->oracle),
`::save_baseline`/`::load_baselines`, `::gt_hash` (label-set fingerprint, reused by
experiment.py), `::RegressionResult`. ? imports `annotation_registry` from
**`backend.annotations`** NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass). CLI
`run_regression.py` exit 0 ok / 1 regress / 2 no-DB / 3 no-baseline = the promotion gate.
181 - `@backend/eval/experiment.py` ? A/B + shadow + **SxS** variant layer (the thin
"experiment" glue per `docs/EXPERIMENT_HARNESS.md`; composes
metrics/calibration/classifier/training/regression ? NO new scoring math).
`::Variant` (segmenter,config_overrides,cue_flags,`min_crossings`[Gate A],`min_players`[Gate
B],classifier_model,feature_names,threshold,merge_gap;
`to_dict`/`from_dict`/`spec_hash`/`is_learned`), `::CONTROL` (pinned zero-gate baseline).
`::VideoCandidates` (name,intervals,features{col-major},gts), `::predict` (gate?learned-
filter?`gemini_refine.merge_overlaps`), `::evaluate_variant` (static / pinned-model / **honest-
LOVO** train-on-others ? mirrors distill_local), `::compare` (labeled **B?A** deltas +
`label_set_hash`), `::compare_unlabeled` (**NEW-footage SxS**: agreed/a_only/b_only via
`match_intervals`, no GT ? the owner's "SxS on new videos"), `::record_experiment` (?
`eval_baselines/experiment_leaderboard.json`), `::load_video_candidates` (DB bridge via
`query_candidate_segments`). ? enabling a gate needs its feature persisted (absent?0.0?fails
gate, warns); learned variant on an unlabeled video needs a pinned `classifier_model`. **No-reg
invariant (tested): all cues OFF ? byte-identical to CONTROL.**
182
183 - `@backend/eval/golden_fixtures.py` ? **video-free regression harness**
(`docs/GOLDEN_REGRESSION_FIXTURES.md`). `::replay_fixture` =
`tracknet_runner.parse_trajectory_csv` ? `distill_local.build_candidates` (PINNED preset) ?
`experiment.VideoCandidates` (5 shuttle cols only) ? `experiment.predict(CONTROL)` ?
`metrics.score`; **CONTROL / pure-stdlib ? no numpy on the replay path** (cross-OS stable; the
learned/numpy path is deliberately excluded). `::freeze_synthetic` (3 committed synthetic Tier-0
fixtures), `freeze_real_fixture` (now DEFINED in the split-out
`@backend/eval/golden_real_fixtures.py` (#233) and re-exported here via PEP-562 `__getattr__`)
(P1/M2: `RefusalError` unless `owned`+`minors_free`+non-blank `license`; opaque random
`fx`<hex> slug, metric-invariant time-origin reset, STRICT `gt_loader`, positive
no-`FORBIDDEN_KEYS`/MD5/source-path scan), `::refresh_snapshot`, `::compare_expected` (exact-int
/ approx-1e-6, NEVER byte-compared), `::load_manifest`/`::load_fixture` (schema_version
dispatch). CLI `--freeze-synthetic`/`--freeze-real`/`--refresh-snapshot`/`--check`. Tests:
`tests/test_golden_fixtures.py` (M1/M3 characterization) + `tests/test_golden_real_fixtures.py`
(P1/M2). CI: the `golden-crossos` job (ubuntu/windows/macOS) runs `--check` + the M1 test. ?
synthetic-tier numbers = plumbing correctness, NOT field quality.
184
185 ### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
186 - `@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector. Pure
numpy + stdlib `wave` (NO scipy); ffmpeg only for extraction. `::HitEvent` (t,strength),

```

```

`::extract_audio` (ffmpeg -> mono 16k PCM WAV), `::load_wav`, `::onset_envelope` (pure-numpy
STFT spectral flux; `band=(lo,hi)` band-limit), `::detect_hits` (adaptive `mean+k.std`;
`k`=sensitivity knob), `::detect_hits_in_video`. ? separate signal ? WASB never modified by
this; per audio-e1-findings recall 100% but discrimination ~2.2x, band-pass null -> NOT adopted
(PR #35).
187 - `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
188
189     ### 2.4 Stitcher / tools / utils
190 - `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::parse_time`
(mirrors gemini.parse_time; ? unparseable -> 0.0), `::get_video_duration`. ? if duration probe
fails (0.0) NO boundary validation; `begin_padding`/`end_padding` ctor defaults 0.5/1.0 but
every live call site (`/api/compile`, CLI trim, `scripts/run_process_and_stitch.py`) threads
`config.json` stitching. `{begin_padding,end_padding}` (typed default 0.5/0.5); temp clips in a
TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses
`print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so
final concat can stream-copy ? change a preset and concat may break. ? optional **branding
watermark** (`stitching.watermark`, default-on): `::watermark_filter` builds a `-vf` (drawtext
via `textfile=` + optional logo `movie/overlay`) applied to the per-clip re-encode only (concat
stays `-c copy`); a failed watermark encode retries the clip WITHOUT it (never nukes the reel,
and logs the ffmpeg stderr reason). Pass `watermark=` to `VideoCompiler(...)` (a
`WatermarkConfig` model or dict; `None` = off). Default font = bundled Apache-2.0
`::BUNDLED_FONT` (`backend/assets/fonts/RobotoSlab-Regular.ttf`) so it renders without
fontconfig; `font_path` overrides.
191 - `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure,
unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::ClipWindow` (rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (?
duplicate compiler's by copy). `read_time` -> backend/eval/annotations.parse_timestamp.
192 - `@tools/generate_highlights.py` ? **the batch highlight RUNNER** (the clean,
orchestrated way to do "make reels for these videos into this folder").
`::generate_highlights_batch(video_paths, out_dir, *, segmenter, local_work_dir, db_path,
force)` + `::atomic_publish` (copy?`os.replace`; never deletes siblings) + `::resolve_videos`.
**Guarantees:** generates LOCALLY then publishes atomically into the dest (Drive-sync-safe ?
never generate inside a Google-Drive folder); **NEVER bulk-deletes** the dest; per-video
`clear_video_data` (idempotent ? no dup-rally accumulation; pairs with the compiler overlap-
merge); resumable (skip published reels unless `--force`); per-video `` - run.log` +
`_batch_run.log` published additively. CLI: `python -m tools.generate_highlights --videos-dir
<dir> --out <dir> --segmenter gemini [--force]` or repeated `--video`. ? WSL detectors
(wasb/tracknet) localize the source first (Drive invisible to /mnt). **Use THIS for batch reels
? never hand-roll a loop that writes into Drive or rm's the folder** (that lost finished reels +
run.logs once).
193 - `@tools/cleanup_caches.py` ? **top-level** `tools/` ops tool (NOT backend runtime).
Dry-run-first cache GC. Pure `::classify` (name,is_dir,location)/`::plan_deletions` (unit-tested
= the destructive decision path) + I/O `::scan_wsl` (du+find; ? arg-globbing NOT a `for` loop ?
the login shell empties loop vars) / `::scan_windows` / `::delete`. CLI: `python -m
tools.cleanup_caches` (report) . `--apply` / `--keep-video <id|stem>` / `--older-than N` /
`--include-wasbcache` / `--summary` (hook nudge). Purges `*_frames`/staged/proxy/handoff/tmp;
keeps CSV/DB/reels. Companion to the runner self-clean (2.2). See `[[cache-hygiene-policy]]`
memory.
194 - `@backend/utils/video.py` ? `::get_video_duration` (ffprobe; ? 0.0 on failure =
"unknown"), `::extract_video_chunk(...,reencode=False,scale_width=None)`. ? copy mode cuts at
nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale_width` requires
`reencode=True`.
195 - `@backend/utils/geometry.py` ? `::get_velocity_normalised` (fraction of frame-width/s;
? raw-px fallback if width<=0; used by yolo_hybrid), `::calculate_iou` (? +1 px convention
inflates IoU), `::get_center/get_distance/get_velocity`.
196
197     ### 2.5 Validators ?
198 - `@backend/pipeline/validators/base.py::VideoValidator` ABC, `::ValidationResult`
(pydantic: validator_name,passed,message,details), `::ValidationRegistry` (ordered LIST ?
registration order = exec order; ? no dedup), `::registry` (singleton). § `validate(video_path,
config, context)` -> `ValidationResult`. `::run_all_with_context` -> (results, context)` (wraps

```



```

each validate in try/except -> failed result not abort); `::run_all` discards context. ? **add a
validator**: subclass + `registry.register(MyValidator())`.
199     - `@backend/pipeline/validators/__init__.py` ? registers in EXEC order: FormatValidator,
ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator,
RelevanceValidator (? producers before consumers ? format_check first; ? import mutates global
registry as side effect).
200     - `@backend/pipeline/validators/format.py::FormatValidator` (`format_check`) ?
**PRODUCER** of `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is
substring `mp4` (so MOV-family passes); external `ffprobe`.
201     - `@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator`
(`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails "Run format_check first" if
absent). `min_fps` 29.9; min res 1280x720.
202     - `@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator`
(`pts_continuity_check`) ? re-probes (no context). ? 10.0s keyframe-gap threshold hardcoded
(comment says 8.0s ? drift, NOT config-driven); any backward jump fails; `<2` keyframes PASSES
(too short).
203     - `@backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator`
(`scene_cut_check`) ? OpenCV. ? `cut_threshold=0.6` hardcoded (NOT config); samples ~2fps, caps
at 100 SAMPLED frames (~first 50s); `max_scene_cuts_per_minute` (0.5) is the config fail
threshold.
204     - `@backend/pipeline/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var <
`min_blur_threshold` (default 100.0 in-file, 20.0 in models/config) = fail; samples 15 frames. ?
magic absolute, not resolution-normalized.
205     - `@backend/pipeline/validators/relevance.py::RelevanceValidator` (`relevance_check`) ?
HSV court-green ratio (? purely color, no geometry despite name); lazy `import backend.sports`;
3-tier config precedence (`validation.relevance` -> sport profile -> hardcoded); ? unknown sport
-> empty cfg, falls back to defaults, does NOT fail.
206
207     ### 2.6 Sports / Providers / Annotations ?
208     - `@backend/sports/base.py::SportProfile` ABC
(`name`, `get_prompt`, `get_relevance_config`);
`@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
`@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton; ?
`register` instantiates profile to read `name`). ? **add a sport**: subclass `SportProfile`,
`sport_registry.register(MyProfile)`. $ prompt emits JSON array `{start_time(float DECIMAL SEC),
end_time, shuttle_exchange_count(int), shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt
actively defends against MM:SS; `get()` returns a fresh instance each call.
209     - `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`);
`@backend/providers/gemini.py::GeminiProvider` (lazy `::client` property; `::MAX_UPLOAD_MB=500`;
upload->poll->generate JSON temp 0->delete); `@backend/providers/__init__.py::provider_registry`
(+`::ProviderRegistry`, auto-registers `gemini`). ? **add a provider**: subclass,
`provider_registry.register('name', MyCls)` (key passed EXPLICITLY); `get(name, api_key,
model_name)`. $ `analyze_video(path, prompt, chunk_duration=0.0, label='')` -> (segments:list,
metrics:dict{upload_time,process_time,gen_time})`. ? blocking 10s poll; returns `([], metrics)`
on ANY failure (empty != error); oversize clip refused LOUDLY (`logger.error` +
`metrics['oversize_skipped']=True` ? callers aggregate into the report's `oversize_clip_skipped`
rule); orphaned remote file possible if process dies mid-run.

```

6. assistant (2026-06-20T08:00:37.426Z)

Now let me look at the stitcher/compiler and the API compile path more closely, plus check for proxy/thumbnail/frame extraction.

7. user (2026-06-20T08:00:38.437Z)

```

1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     logger = logging.getLogger(__name__)
9
10    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the

```

```

overlay
11     # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12     # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
13     # backend/pipeline/stitcher/, so parents[2] == backend/.
14     _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16     class VideoCompiler:
17         def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                     watermark: Optional[Any] = None):
19             self.output_dir = output_dir
20             self.end_padding = end_padding
21             self.begin_padding = begin_padding
22             # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23             # normalized to a dict (or None) so this module stays config-package-agnostic.
24             self.watermark = self._normalize_watermark(watermark)
25             os.makedirs(output_dir, exist_ok=True)
26
27         @staticmethod
28         def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29             if watermark is None:
30                 return None
31             if hasattr(watermark, "model_dump"):
32                 return watermark.model_dump()
33             if isinstance(watermark, dict):
34                 return dict(watermark)
35             return None
36
37         @staticmethod
38         def _ff_quote(path: str) -> str:
39             """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40             filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41             slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43             The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44             option parser treats ':' as the option separator, so a Windows drive-letter
45             path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
46             the encode fails with "Invalid argument". This silently disabled the
47             on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48             return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:");
49
50         @staticmethod
51         def _parse_time(val: Union[int, float, str]) -> float:
52             """Normalizes a timestamp value to float seconds.
53             Handles: float/int (pass-through), plain numeric strings ("12.4"),
54             and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55             This mirrors the parse_time logic used in gemini.py to ensure
56             the stitcher can always consume whatever format the indexer produced."""
57             if isinstance(val, (int, float)):
58                 return float(val)
59             if isinstance(val, str):
60                 val = val.strip()
61                 if ":" in val:
62                     # Handle MM:SS, HH:MM:SS, etc.
63                     parts = val.split(":")
64                     parts.reverse()
65                     total = 0.0
66                     for i, p in enumerate(parts):
67                         try:
68                             total += float(p) * (60 ** i)
69                         except ValueError:
70                             pass

```

```

71         return total
72     try:
73         return float(val)
74     except ValueError:
75         logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
76         return 0.0
77         logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
78         return 0.0
79
80     @staticmethod
81     def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
float]]:
82         """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
83
84         A highlight reel must never replay the same footage. The same rally can land in
85         the segment list more than once ? e.g. two detector runs accumulated for one
86         video (add_video is an UPSERT, so re-running without a prune leaves both sets),
87         or
88         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
89         rally
90         twice. Merging any range that overlaps or abuts the previous one collapses true
91         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
92         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
93         """
94         parsed = []
95         for raw_s, raw_e in segments:
96             s = VideoCompiler._parse_time(raw_s)
97             e = VideoCompiler._parse_time(raw_e)
98             if e > s:
99                 parsed.append((s, e))
100         parsed.sort()
101         merged: List[Tuple[float, float]] = []
102         for s, e in parsed:
103             if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
104                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
105             else:
106                 merged.append((s, e))
107         return merged
108
109     def _get_video_duration(self, video_path: str) -> float:
110         """Probes the video file to get its total duration in seconds.
111         Returns 0.0 if it cannot be determined."""
112         try:
113             result = subprocess.run(
114                 [
115                     "ffprobe", "-v", "error",
116                     "-show_entries", "format=duration",
117                     "-of", "default=noprint_wrappers=1:nokey=1",
118                     video_path
119                 ],
120                 capture_output=True, text=True, check=True
121             )
122             return float(result.stdout.strip())
123         except Exception as e:
124             logger.warning(f"Could not determine video duration via ffprobe: {e}")
125             return 0.0
126
127     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
128         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
129         and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
130         watermarking is disabled or has nothing to draw. The concat stage stays -c copy
131         because the mark is already baked into every clip identically."""
132         wm = self.watermark

```

```

131         if not wm or not wm.get("enabled"):
132             return None
133
134         text = (wm.get("text") or "").strip()
135         logo_path = (wm.get("logo_path") or "").strip()
136         if logo_path and not os.path.exists(logo_path):
137             logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
overlay.")
138             logo_path = ""
139         if not text and not logo_path:
140             return None
141
142         margin = int(wm.get("margin", 24))
143         opacity = float(wm.get("opacity", 0.85))
144         position = str(wm.get("position", "bottom-right"))
145         # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
146         dt_xy = {
147             "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
148             "bottom-left": f"x={margin}:y=h-th-{margin}",
149             "top-right": f"x=w-tw-{margin}:y={margin}",
150             "top-left": f"x={margin}:y={margin}",
151         }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
152         ov_xy = {
153             "bottom-right": f"W-w-{margin}:H-h-{margin}",
154             "bottom-left": f"{margin}:H-h-{margin}",
155             "top-right": f"W-w-{margin}:{margin}",
156             "top-left": f"{margin}:{margin}",
157         }.get(position, f"W-w-{margin}:H-h-{margin}")
158
159         drawtext = ""
160         if text:
161             ratio = float(wm.get("font_size_ratio") or 0.035)
162             divisor = max(1, round(1.0 / ratio))
163             # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
164             tf_path = os.path.join(tmp_dir, "watermark.txt")
165             with open(tf_path, "w", encoding="utf-8") as fh:
166                 fh.write(text)
167             # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
168             # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
169             font_path = (wm.get("font_path") or "").strip()
170             if not font_path and _BUNDLED_FONT.exists():
171                 font_path = str(_BUNDLED_FONT)
172             if font_path:
173                 font_opt = f"fontfile='{self._ff_quote(font_path)}'"
174             else:
175                 font_opt = f"font='{wm.get('font', 'Sans')}'"
176             box = ""
177             if wm.get("box", True):
178                 box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
179             drawtext = (
180                 f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
181                 f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
182             )
183
184         if logo_path and drawtext:
185             return
186         f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
187         if logo_path:
188             return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
189         return drawtext
190
191     def _trim_one_segment(

```

```

191         self,
192         idx: int,
193         raw_start: Union[int, float, str],
194         raw_end: Union[int, float, str],
195         segments: List[Tuple[float, float]],
196         video_duration: float,
197         tmp_dir: str,
198         raw_video_path: str,
199         watermark_filter: Optional[str],
200     ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
201         """Trim a single segment into its own clip file (extracted verbatim from the
202         per-segment body of compile_highlights). Returns (clip_path, skip_record): on
203         success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
204         None and the (idx, start, end, reason) record to record in skipped_segments."""
205         # Normalize timestamps to float seconds ? handles float, int,
206         # plain numeric strings, and colon-separated formats like "MM:SS"
207         start = self._parse_time(raw_start)
208         end = self._parse_time(raw_end)
209         segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"
210
211         # Validate the segment times
212         if start < 0:
213             logger.warning(f"{segment_label}: start_time is negative ({start:.2f}s).
Clamping to 0.")
214             start = 0.0
215
216         if start >= end:
217             logger.error(f"{segment_label}: start_time ({start:.2f}s) >= end_time
({end:.2f}s). Skipping invalid segment.")
218             return None, (idx, start, end, "start >= end")
219
220         # Check if segment extends beyond video duration
221         if video_duration > 0:
222             if start >= video_duration:
223                 logger.error(f"{segment_label}: start_time ({start:.2f}s) is beyond the
video duration ({video_duration:.2f}s). "
224                             f"This segment cannot be extracted ? the video ran out.
Skipping.")
225                 return None, (idx, start, end, "start beyond video end")
226
227             if end > video_duration:
228                 original_end = end
229                 end = video_duration
230                 logger.warning(f"{segment_label}: end_time ({original_end:.2f}s) exceeds
video duration ({video_duration:.2f}s). "
231                             f"Clamping end_time to {video_duration:.2f}s. The segment
may be truncated.")
232
233             clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
234
235             # Precise sub-second trim using fast re-encoding to preserve stream headers
236             padded_start = max(0.0, start - self.begin_padding)
237             padded_end = end + self.end_padding
238
239             # Clamp padded_end to video duration if known
240             if video_duration > 0 and padded_end > video_duration:
241                 padded_end = video_duration
242                 logger.info(f"{segment_label}: Padded end ({end + self.end_padding:.2f}s)
exceeds video duration. "
243                             f"Clamping to {video_duration:.2f}s (end padding partially
applied).")
244
245             trim_duration = padded_end - padded_start
246             logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")

```

```

247
248     base_cmd = [
249         "ffmpeg", "-y",
250         "-ss", str(round(padded_start, 3)),
251         "-to", str(round(padded_end, 3)),
252         "-i", raw_video_path,
253         "-c:v", "libx264",
254         "-preset", "superfast",
255         "-crf", "22",
256         "-c:a", "aac",
257         "-b:a", "128k",
258     ]
259     vf_args = ["-vf", watermark_filter] if watermark_filter else []
260
261     # Try with the watermark first; if that encode fails (e.g. a bad font/logo
262     # path), retry once WITHOUT it so a branding misconfig can never nuke the
263     # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
264     attempts = [vf_args, []] if vf_args else [[]]
265     produced = False
266     last_exc: Optional[subprocess.CalledProcessError] = None
267     cmd = base_cmd + [clip_path]
268     for attempt_vf in attempts:
269         cmd = base_cmd + attempt_vf + [clip_path]
270         try:
271             subprocess.run(cmd, capture_output=True, check=True)
272         except subprocess.CalledProcessError as e:
273             last_exc = e
274             if attempt_vf and vf_args:
275                 _err = (e.stderr.decode(errors="replace").strip()[-300:])
276                 if getattr(e, "stderr", None) else ""
277                 logger.warning(
278                     f"{segment_label}: watermark encode failed; retrying without it.
279 "
280                     f"ffmpeg: {_err or '(no stderr captured)'}"
281                 )
282                 continue
283             if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
284                 produced = True
285                 if not attempt_vf and vf_args:
286                     logger.warning(f"{segment_label}: encoded WITHOUT the watermark (the
287 watermark filter failed).")
288                 break
289                 last_exc = None # ffmpeg succeeded but produced nothing
290                 break
291     if produced:
292         return clip_path, None
293     elif last_exc is not None:
294         stderr_msg = last_exc.stderr.decode(errors='replace').strip()
295         logger.error(f"{segment_label}: FFmpeg trim failed.")
296         print(f"  Command: {' '.join(cmd)}")
297         print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to avoid
298 flooding
299         return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
300     else:
301         logger.error(f"{segment_label}: FFmpeg exited successfully but output clip
302 is missing or empty. Skipping.")
303         return None, (idx, start, end, "empty output clip")
304
305     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
306 float]], output_filename: str) -> str:
307         """
308         Extracts selected segments from a raw video and stitches them together
309         into a seamless highlights file, applying end_padding to prevent abrupt endings.

```

```

307         """
308         if not segments:
309             raise ValueError("No highlight segments provided to compile.")
310
311         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
312         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
313         original_count = len(segments)
314         segments = self._merge_overlapping(segments)
315         if len(segments) < original_count:
316             logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
-> %d.",
317                         original_count - len(segments), original_count, len(segments))
318         if not segments:
319             raise ValueError("No valid (start < end) highlight segments after merge.")
320
321         if not os.path.exists(raw_video_path):
322             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
323
324         # Defense-in-depth: never let the output name escape output_dir (the API also
325         # validates this at the request boundary).
326         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
327
328         # Probe video duration so we can detect out-of-range segments
329         video_duration = self._get_video_duration(raw_video_path)
330         if video_duration > 0:
331             logger.info(f"Source video duration: {video_duration:.2f}s")
332         else:
333             logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")
334
335         # We will create temporary clips and stitch them using FFmpeg concat demuxer
336         temp_clips = []
337         skipped_segments = []
338
339         # Create a temp directory within output_dir
340         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
341             logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
342             # Build the (optional) watermark filtergraph once; it is applied during
every
343             # per-clip re-encode so the mark is baked into each clip identically.
344             watermark_filter = self._watermark_filter(tmp_dir)
345             if watermark_filter:
346                 logger.info("[watermark] applying branding overlay to each clip.")
347             for idx, (raw_start, raw_end) in enumerate(segments):
348                 clip_path, skip_record = self._trim_one_segment(
349                     idx, raw_start, raw_end, segments, video_duration,
350                     tmp_dir, raw_video_path, watermark_filter,
351                 )
352                 if clip_path is not None:
353                     temp_clips.append(clip_path)
354                 if skip_record is not None:
355                     skipped_segments.append(skip_record)
356
357             # Report summary of skipped segments
358             if skipped_segments:
359                 logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
360                 for seg_idx, seg_start, seg_end, reason in skipped_segments:
361                     logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
362                 logger.info("")
363
364             if not temp_clips:

```

```

365         raise RuntimeError(
366             f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
367             f"All {len(segments)} segments were skipped. "
368             f"Check the errors above for details. "
369             f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
370         )
371
372     logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
373
374     # Create a list file for FFmpeg concat demuxer
375     list_file_path = os.path.join(tmp_dir, "clips.txt")
376     with open(list_file_path, "w") as f:
377         for clip in temp_clips:
378             # Escape single quotes in path
379             safe_path = clip.replace("'", "'\\'")
380             f.write(f"file '{safe_path}'\n")
381
382     logger.info("Stitching clips together...")
383     # Stitch them together without re-encoding (since they were encoded with
identical settings)
384     concat_cmd = [
385         "ffmpeg", "-y",
386         "-f", "concat",
387         "-safe", "0",
388         "-i", list_file_path,
389         "-c", "copy",
390         output_path
391     ]
392
393     try:
394         subprocess.run(concat_cmd, capture_output=True, check=True)
395
396         # Final verification
397         if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
398             raise RuntimeError(
399                 f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
400                 f"but output file is missing or empty: {output_path}"
401             )
402
403         output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
404         logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
405         return output_path
406     except subprocess.CalledProcessError as e:
407         stderr_msg = e.stderr.decode(errors='replace').strip()
408         raise RuntimeError(
409             f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
410             f"Command: {' '.join(concat_cmd)}\n"
411             f"FFmpeg stderr: {stderr_msg[-500:]}"
412         ) from e
413

```

8. user (2026-06-20T08:00:39.097Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     import uvicorn
7     from dotenv import load_dotenv
8
9     load_dotenv()

```



```

10
11     # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
12     logging.basicConfig(
13         level=logging.INFO,
14         format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
15         datefmt='%H:%M:%S'
16     )
17     logger = logging.getLogger(__name__)
18     from fastapi import FastAPI, HTTPException
19     from fastapi.middleware.cors import CORSMiddleware
20     from pydantic import BaseModel
21     from typing import List, Dict, Any, Optional
22
23     from fastapi.staticfiles import StaticFiles
24     from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
25     from backend.pipeline.validators.base import registry
26     from backend.infrastructure.database import Database
27     from backend.pipeline.segmenters.base import segmenter_registry
28     from backend.pipeline.stitcher.compiler import VideoCompiler
29     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
30     from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
31     from backend.config import load_config as load_app_config, AppConfig, StitchingConfig #
new typed config
32     from backend.results import Failure # standardized error/result contract
33     from backend.reporting import emit_indexer_report
34
35     app = FastAPI(title="Sports Highlight Indexer API")
36
37     # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is
38     # rejected by browsers per the fetch spec and is an unsafe default to carry into the
39     # auth/tenancy work; this tool uses no cookies, so credentials stay off.
40     app.add_middleware(
41         CORSMiddleware,
42         allow_origins=["*"],
43         allow_credentials=False,
44         allow_methods=["*"],
45         allow_headers=["*"],
46     )
47
48     # Serves static frontend files directly (now under api/ per approved structure)
49     os.makedirs("backend/api/static", exist_ok=True)
50     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
51
52     @app.get("/", response_class=HTMLResponse)
53     def serve_index():
54         index_path = "backend/api/static/index.html"
55         if os.path.exists(index_path):
56             with open(index_path, "r", encoding="utf-8") as f:
57                 return f.read()
58         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
59
60     # Global variables initialized at startup
61     db_instance: Optional[Database] = None
62     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
63
64     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
65     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
66     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
67     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH

```

```

68     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
69     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
70     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
71     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
72         JobWaiting,
73         _arm_wake_timer,
74         _drain_due_jobs,
75         _get_executor,
76         _job_to_request,
77         _MAX_DRAIN_WAKE_SEC,
78         _run_claimed_job,
79         _schedule_next_drain_if_waiting,
80     )
81
82     # Legacy thin wrapper for any remaining direct calls during transition.
83     # New code should import load_app_config / AppConfig from backend.config directly.
84     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
85         cfg = load_app_config(config_path)
86         return cfg.model_dump(mode="json")
87
88     # --- API Models ---
89     class ProcessRequest(BaseModel):
90         video_path: str
91         player_pool: Optional[List[str]] = None
92         max_frames: Optional[int] = None
93         segmenter: Optional[str] = None
94
95     class QueryRequest(BaseModel):
96         video_id: str
97         server: Optional[str] = None
98         receiver: Optional[str] = None
99         duration_min: Optional[float] = None
100         event_type: Optional[str] = None
101
102     class CompileRequest(BaseModel):
103         video_id: str
104         segment_ids: List[int]
105         output_filename: str
106
107     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
108         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
109
110         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
111         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
112         and
113         keep the prior good results intact ? a failed/empty re-run never destroys prior
114         data.
115         """
116         if segments:
117             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
118         else:
119             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
120
121     # --- API Endpoints ---
122     @app.get("/api/config")
123     def get_config():
124         return global_config
125
126     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
127         """The full hash ? validate ? segment ? report pipeline for one video.
128
129         This is the former synchronous /api/process body, unchanged in behavior, now run on
130         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict

```

```

130     the sync endpoint used to return (UI compatibility); the per-video observability
131     report is still emitted in `finally:` and grafted as response["reports"].
132
133     `progress` is an optional callable(stage: str) used to surface coarse job progress.
134     Raises on unexpected internal errors ? the caller records those as a job failure
135     (after this function has already restored the pre-run segments, see below).
136     """
137     notify = progress or (lambda stage: None)
138
139     notify("hashing")
140     video_id = compute_video_id(req.video_path)
141     was_reingest = db_instance.get_video(video_id) is not None
142
143     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
144     notify("validating")
145     logger.info(f"Running validations for {req.video_path}...")
146     val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
147     failures = [r for r in val_results if not r.passed]
148
149     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
150     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
151     seg_name = req.segmenter or default_seg
152     segmenter_actual = None
153     segmentation_ran = False
154     response: Optional[Dict[str, Any]] = None
155     try:
156         if failures:
157             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
158             # status; it no longer destroys the video's prior good segments.
159             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
160             response = {
161                 "video_id": video_id,
162                 "status": "failed",
163                 "message": "Video failed validation checks.",
164                 "results": [r.model_dump() for r in val_results],
165             }
166             return response
167
168     # 2. Run segmenter & indexer
169     logger.info("[API] Video passed checks. Segmenting and indexing...")
170     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
171
172     segmenter_cls = segmenter_registry.get(seg_name)
173     if not segmenter_cls:
174         # Submit-time validation already 400s unknown names; this is the defensive
175         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
176         available = list(segmenter_registry.list_available().keys())
177         response = {
178             "video_id": video_id,
179             "status": "failed",
180             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
181             "results": [r.model_dump() for r in val_results],
182             "play_segments": [],
183         }
184         return response
185
186     logger.info(f"[API] Using segmenter: {seg_name}")
187     notify(f"segmenting ({seg_name})")

```

```

188         segmenter_actual = seg_name
189         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
190         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
191         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
192         segmenter = segmenter_cls(db_instance, global_config)
193         try:
194             result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
195         except Exception:
196             # A raising segmenter must not leave half-written rows: restore the pre-run
197             # state (same destroy-after-confirm contract as the Failure branch), then
let
198             # the job worker record the failure.
199             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
200             raise
201         segmentation_ran = True
202
203         if isinstance(result, Failure):
204             logger.error(f"[API] Segmenter failed: {result.message}")
205             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
206             response = {
207                 "video_id": video_id,
208                 "status": "failed",
209                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
210                 "results": [r.model_dump() for r in val_results],
211                 "play_segments": [],
212             }
213             return response
214
215         segments = result.value if hasattr(result, "value") else result
216         notify("finalizing")
217         _finalize_reingest(video_id, segments, play_wm, cand_wm)
218
219         response = {
220             "video_id": video_id,
221             "status": "success",
222             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
223             "results": [r.model_dump() for r in val_results],
224             "play_segments": segments,
225         }
226         return response
227     finally:
228         # Always emit the per-video observability report (next to the video, or to
229         # report.output_dir). Never raises. Surface the paths in the response.
230         paths = emit_indexer_report(
231             db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
232             val_results=val_results, val_context=val_context,
233             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
234             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
235         )
236         if paths and isinstance(response, dict):
237             response["reports"] = paths
238
239
240     @app.post("/api/process", status_code=202)
241     def process_video(req: ProcessRequest):
242         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
243
244         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
245         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
246         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the

```

```

247     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
248     """
249     allowed_root = getattr(getattr(global_config, "security", None),
250 "allowed_video_root", None)
251     try:
252         validate_input_path(req.video_path, allowed_root=allowed_root)
253     except ValueError as e:
254         raise HTTPException(status_code=400, detail=str(e)) from e
255     if not os.path.exists(req.video_path):
256         raise HTTPException(status_code=404, detail=f"Video file not found at
257 '{req.video_path}'")
258     if req.segmenter and not segmenter_registry.get(req.segmenter):
259         available = list(segmenter_registry.list_available().keys())
260         raise HTTPException(
261             status_code=400,
262             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
263 {available}"
264         )
265     job_id = uuid.uuid4().hex
266     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
267 req.player_pool, req.max_frames):
268         raise HTTPException(status_code=500, detail="Failed to persist job record.")
269     _get_executor().submit(_drain_due_jobs)
270     return JSONResponse(
271         status_code=202,
272         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
273     )
274
275 @app.get("/api/jobs/{job_id}")
276 def get_job(job_id: str):
277     """Job state: {status, progress, result, reports}. `status` = execution state
278 (queued|running|done|failed); the pipeline verdict is result["status"]."""
279     job = db_instance.get_job(job_id)
280     if not job:
281         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
282     result = job.get("result")
283     return {
284         "job_id": job["id"],
285         "status": job["status"],
286         "progress": job.get("progress"),
287         "video_id": job.get("video_id"),
288         "error": job.get("error"),
289         "result": result,
290         "reports": (result or {}).get("reports"),
291         "created_at": job.get("created_at"),
292         "started_at": job.get("started_at"),
293         "finished_at": job.get("finished_at"),
294     }
295
296 # --- Chunked upload (B2, PR-2) -----
297 # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
298 in
299 # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
300 via
301 # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
302 # sequential-part logic, and abort cleanup ? is unchanged.
303 from backend.api import uploads as uploads_api
304
305 app.include_router(uploads_api.router)
306
307 # --- Highlight download (B3, PR-2) -----

```

```

307     @app.get("/api/highlights/{video_id}/{filename}")
308     def download_highlight(video_id: str, filename: str):
309         """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
310         (which only writes into output_dir; the browser previously got back an unreachable
311         server-local path)."""
312         try:
313             name = safe_output_filename(filename)
314         except ValueError as e:
315             raise HTTPException(status_code=400, detail=str(e)) from e
316         if not db_instance.get_video(video_id):
317             raise HTTPException(status_code=404, detail="Indexed video record not found.")
318         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
319         path = os.path.join(output_dir, name)
320         try:
321             path = resolve_under_root(path, output_dir)
322         except ValueError as e:
323             raise HTTPException(status_code=400, detail=str(e)) from e
324         if not os.path.isfile(path):
325             raise HTTPException(status_code=404,
326                                 detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
327         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
328         return FileResponse(path, media_type=media, filename=name)
329
330
331     @app.post("/api/query")
332     def query_play_segments(req: QueryRequest):
333         filters = {
334             "server": req.server,
335             "receiver": req.receiver,
336             "duration_min": req.duration_min,
337             "event_type": req.event_type
338         }
339         segments = db_instance.query_play_segments(req.video_id, filters)
340         return {"play_segments": segments}
341
342     @app.post("/api/compile")
343     def compile_highlights(req: CompileRequest):
344         video = db_instance.get_video(req.video_id)
345         if not video:
346             raise HTTPException(status_code=404, detail="Indexed video record not found.")
347
348         # Retrieve matching play segments from db
349         all_segments = db_instance.query_play_segments(req.video_id, {})
350         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
351
352         if not matched_segments:
353             raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
354
355         # Reject path traversal / absolute output names (os.path.join would otherwise let an
356         # absolute or '..'-laden output_filename write anywhere on disk).
357         try:
358             out_name = safe_output_filename(req.output_filename)
359         except ValueError as e:
360             raise HTTPException(status_code=400, detail=str(e)) from e
361
362         # The configured shot-count floor (stitching.min_shot_count) applies on the API
363         # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
364         # metadata are exempt: the floor filters known counts only, so explicitly
365         # requested manual/legacy segments can't silently vanish under the default floor.
366         stitching = getattr(global_config, "stitching", None) or StitchingConfig()

```

```

367     kept = []
368     for s in matched_segments:
369         meta = s.get("metadata") or {}
370         sc = meta.get("shot_count") if isinstance(meta, dict) else None
371         try:
372             if sc is not None and int(sc) < stitching.min_shot_count:
373                 continue
374             except (TypeError, ValueError):
375                 pass # unparseable count -> treat as unknown, keep
376             kept.append(s)
377         if len(kept) < len(matched_segments):
378             logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
379                         f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
380             if not kept:
381                 raise HTTPException(status_code=400,
382                                     detail=f"All {len(matched_segments)} matching segments fall
below "
383                                     f"stitching.min_shot_count={stitching.min_shot_count}.")
384
385             # Extract start/end time pairs
386             time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
387
388             # Run compiler with the configured stitching paddings + optional branding watermark
389             output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
390             or "output"
391             compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
392                                     end_padding=stitching.end_padding,
393                                     watermark=stitching.watermark)
394             try:
395                 out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
396                 return {"status": "success", "filepath": out_path}
397             except Exception as e:
398                 raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
399
400             # --- CLI Debug Utility & Orchestration ---
401             def run_cli_diagnose_clip(video_path: str, timestamp: float):
402                 """CLI debugging utility to examine segment properties around a timestamp."""
403                 print("-" * 60)
404                 print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
405                 print("-" * 60)
406
407                 cfg = load_config() # legacy wrapper still works, returns dict for now
408
409                 # 1. Validation check diagnostics
410                 print("[DIAGNOSTIC] Running validation framework...")
411                 results = registry.run_all(video_path, cfg)
412                 for r in results:
413                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
414                     print(f"    {status_symbol} {r.validator_name}: {r.message}")
415                     if r.details:
416                         print(f"        Details: {r.details}")
417
418                 # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
419                 print("-" * 60)
420                 print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
421                 import cv2
422                 cap = cv2.VideoCapture(video_path)
423                 if not cap.isOpened():
424                     print("[FAIL] OpenCV could not open video.")

```

```

425     fps = cap.get(cv2.CAP_PROP_FPS)
426     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
427
428     start_frame = max(0, int((timestamp - 3.0) * fps))
429     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
430
431     # Measure frame-by-frame changes in sample region
432     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
433     prev_gray = None
434     motions = []
435
436     for _ in range(start_frame, end_frame):
437         ret, frame = cap.read()
438         if not ret:
439             break
440         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
441         gray = cv2.GaussianBlur(gray, (21, 21), 0)
442
443         if prev_gray is not None:
444             diff = cv2.absdiff(prev_gray, gray)
445             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
446             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
447             motions.append(motion_ratio)
448             prev_gray = gray
449
450     cap.release()
451
452     if motions:
453         avg_motion = sum(motions) / len(motions)
454         # Support both legacy dict (from wrapper) and future direct model
455         if hasattr(cfg, "indexing"):
456             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
457         else:
458             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
459         print(f" Sample Frame Count: {len(motions)}")
460         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
461         if avg_motion > threshold:
462             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
463         else:
464             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
465     else:
466         print(" [ERROR] No visual frames were extracted in the sample range.")
467
468     print("=" * 60)
469
470     def main():
471         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
472         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
473         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
474         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
475         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
476         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
477         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
478         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N

```



```

sub-sampled frames for debugging")
479
480     available_segmenters = list(segmenter_registry.list_available().keys())
481     parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
482     parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
483     parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
484     parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
485
486     args = parser.parse_args()
487
488     if args.setup:
489         # Invoke the diagnostics/bootstrap script (renamed from the old root
490         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
491         # build system). Resolve its path from this file so `indexer --setup`
492         # works regardless of the caller's cwd.
493         import subprocess
494         doctor_path = os.path.join(
495             os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
496             "scripts", "doctor.py",
497         )
498         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
499         subprocess.run([sys.executable, doctor_path])
500         return
501
502     if args.trim_start is not None and args.trim_end is not None:
503         if not args.video_path:
504             print("[ERROR] Please provide --video-path to extract a segment.")
505             return
506
507         # Determine output path
508         out_filename = args.trim_output or "trimmed_segment.mp4"
509         if os.path.isabs(out_filename):
510             out_path = out_filename
511             out_dir = os.path.dirname(out_path)
512             out_name = os.path.basename(out_path)
513         else:
514             out_dir = args.output_dir
515             out_path = os.path.join(out_dir, out_filename)
516             out_name = out_filename
517
518         os.makedirs(out_dir, exist_ok=True)
519         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
520
521         # global_config isn't initialized this early; load the typed config so the trim
522         # clip carries the same configured stitching paddings + watermark as a real
523         # highlight (watermark default-on).
524         stitching = load_app_config().stitching
525         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
526                                 end_padding=stitching.end_padding,
527                                 watermark=stitching.watermark)
528         try:
529             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
530             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
531         except Exception as e:
532             print(f"[ERROR] Failed to compile trimmed segment: {e}")
533         return
534
535     if args.debug_clip is not None:
536         if not args.video_path:

```

```

536         print("[ERROR] Please provide --video-path to debug a specific clip.")
537         return
538     run_cli_diagnose_clip(args.video_path, args.debug_clip)
539     return
540
541     # Initialize Global Config and DB
542     global global_config, db_instance
543     global_config = load_app_config() # returns typed AppConfig
544
545     # The CLI --output-dir overrides the configured output directory. It now lives
546     # on the typed model (OutputConfig.output_dir), so every consumer ? including
547     # /api/compile ? reads the same value instead of a write-only runtime hack.
548     global_config.output.output_dir = args.output_dir
549     os.makedirs(global_config.output.output_dir, exist_ok=True)
550
551     db_path = os.path.join(global_config.output.output_dir,
552 global_config.output.db_name)
553     db_instance = Database(db_path)
554
555     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
556     # the executor is in-process. Mark them failed so pollers see a terminal state.
557     swept = db_instance.fail_stale_jobs()
558     if swept:
559         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
560 failed.")
561
562     # CLI processing mode (no web UI needed)
563     if args.video_path and not args.server:
564         print(f"[CLI] Processing video file: {args.video_path}")
565         if not os.path.exists(args.video_path):
566             print(f"[FAIL] Video file not found: {args.video_path}")
567             return
568
569         video_id = compute_video_id(args.video_path)
570         was_reingest = db_instance.get_video(video_id) is not None
571
572         # Validate (with-context variant surfaces ffmpeg_meta for the report)
573         print("[CLI] Validating...")
574         val_results, val_context = registry.run_all_with_context(args.video_path,
575 global_config)
576         failures = [r for r in val_results if not r.passed]
577
578         # Save video status
579         status = "failed" if failures else "processed"
580         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
581 in val_results])
582
583         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
584 "default_segmenter", "motion")
585         segmenter_actual = None
586         segmentation_ran = False
587         try:
588             print("\n" + "="*40)
589             print("VALIDATION SUMMARY")
590             print("="*40)
591             for r in val_results:
592                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
593                 print(f"{status_symbol} {r.validator_name}: {r.message}")
594             print("="*40 + "\n")
595
596             if failures:
597                 print("[FAIL] Video failed checks. Indexing halted.")
598                 return
599
600         # Index

```

```

596         segmenter_cls = segmenter_registry.get(seg_name)
597     if not segmenter_cls:
598         print(f"[FAIL] Segmenter '{seg_name}' not found.")
599         return
600
601     print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
602     segmenter_actual = seg_name
603     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
604     segmenter = segmenter_cls(db_instance, global_config)
605     result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
606     segmentation_ran = True
607
608     if isinstance(result, Failure):
609         print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
610         print("[FAIL] Indexing halted for video.")
611         segments = []
612     else:
613         segments = result.value if hasattr(result, "value") else result
614         print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
615         # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
616         _finalize_reingest(video_id, segments, play_wm, cand_wm)
617     finally:
618         # Always emit the per-video observability report. Never raises.
619         paths = emit_indexer_report(
620             db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
621             val_results=val_results, val_context=val_context,
622             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
623             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
624         )
625     if paths:
626         print(f"[REPORT] Technical report : {paths.get('technical_md')}")
627         print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
628         print(f"[REPORT] Machine JSON      : {paths.get('json')}")
629     return
630
631     # Start Local Server Mode
632     if args.server or not args.video_path:
633         print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
634         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
635         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
636         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
637         _get_executor().submit(_drain_due_jobs)
638         uvicorn.run(app, host="127.0.0.1", port=args.port)
639
640     if __name__ == "__main__":
641         main()
642

```

9. assistant (2026-06-20T08:00:42.969Z)

Now I have the full picture of the DB schema, the compile/stitch path, and the API. Let me check what metadata the live segmenters actually persist into `play\_segments.metadata` and `events`, plus look for any proxy/thumbnail/frame extraction for playback. Let me check the gemini segmenter (the default), the sports prompt schema, and search for proxy/thumbnail.

10. user (2026-06-20T08:00:44.326Z)

```

1     import os

```

```

2     import time
3     import logging
4     import subprocess
5     import concurrent.futures
6     from typing import List, Dict, Any, Tuple, Callable
7     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
8     from backend.utils.video import get_video_duration, extract_video_chunk
9     from backend.sports import sport_registry
10    from backend.providers import provider_registry
11    from backend.reporting import run_signals
12
13    logger = logging.getLogger(__name__)
14
15
16    def _chunk_workdir(video_id: str) -> str:
17        """Per-video chunk staging dir under output/chunks/<video_id>.
18
19        Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
20        otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
21        in-flight work. A per-video subdir means each run only ever creates/cleans its own.
22        """
23        return os.path.join("output", "chunks", video_id)
24
25
26    def _parse_time(val) -> float:
27        """Converts a timestamp value to total float seconds.
28        Handles float/int pass-through, plain numeric strings, and
29        colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
30        when unexpected formats are encountered."""
31        if isinstance(val, (int, float)):
32            return float(val)
33        if isinstance(val, str):
34            val = val.strip()
35            if ":" in val:
36                # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal seconds.
37                # Convert it, but warn so the issue is visible in logs.
38                parts = val.split(":")
39                parts.reverse()
40                total = 0.0
41                for i, p in enumerate(parts):
42                    try:
43                        total += float(p) * (60 ** i)
44                    except ValueError:
45                        pass
46                logger.warning(f"Timestamp '{val}' was in colon-separated format
47(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
48                return total
49            try:
50                return float(val)
51            except ValueError:
52                logger.warning(f"Could not parse timestamp string '{val}' as a number.
53Defaulting to 0.0.")
54            return 0.0
55        logger.warning(f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
56Defaulting to 0.0.")
57        return 0.0
58
59
60    class GeminiPlaySegmenter(BasePlaySegmenter):
61        @property
62        def name(self) -> str:
63            return "gemini"
64
65        @property
66        def description(self) -> str:

```

```

64         return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
65
66     def _resolve_provider(self, model_name: str):
67         """Resolve the sport profile, provider, and API key (the process_video
preamble).
68
69         Returns (sport_profile, provider) on success, or None to signal any of the
70         early-return cases (unknown sport, missing API key, unknown provider) ? the
71         caller returns [] for None.
72         """
73         # Get sport profile config
74         sport_name = self.config.get("sport", "badminton")
75         try:
76             sport_profile = sport_registry.get(sport_name)
77         except KeyError as e:
78             logger.error(str(e))
79             return None
80
81         # Get AI provider and model config
82         idx_cfg = self.config.get("indexing", {})
83         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
84
85         # Get appropriate API key based on the provider
86         api_key_env_var = f"{provider_name.upper()}_API_KEY"
87         api_key = os.environ.get(api_key_env_var)
88         if not api_key:
89             logger.error(
90                 f"{api_key_env_var} environment variable is not set! "
91                 f"To run the {provider_name} segmenter, set your API key. "
92                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
93             )
94             return None
95
96         try:
97             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
98         except KeyError as e:
99             logger.error(str(e))
100             return None
101
102         return sport_profile, provider
103
104     def _maybe_trim_video(self, video_id: str, video_path: str) -> Tuple[str,
Callable[[], None]]:
105         """Optionally trim the video to indexing.debug_duration seconds (debug feature).
106
107         Returns (video_to_upload, cleanup_temp_files) where cleanup_temp_files removes
any
108         temp trimmed file created here.
109         """
110         # A: Check for debug_duration to trim the video first
111         debug_duration = self.config.get("indexing", {}).get("debug_duration")
112         video_to_upload = video_path
113         is_trimmed = False
114         temp_trimmed_path = None
115
116         if debug_duration:
117             logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
118             temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
119             os.makedirs("output", exist_ok=True)
120             if os.path.exists(temp_trimmed_path):
121                 try:

```

```

122         os.remove(temp_trimmed_path)
123     except Exception:
124         pass
125
126     cmd = [
127         "ffmpeg", "-y",
128         "-ss", "0",
129         "-t", str(debug_duration),
130         "-i", video_path,
131         "-c", "copy",
132         temp_trimmed_path
133     ]
134     logger.debug(f"Running command: {' '.join(cmd)}")
135     try:
136         subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
137 stderr=subprocess.DEVNULL)
138     except Exception:
139         logger.info(f"Trimmed video successfully created at:
140 {temp_trimmed_path}")
141         video_to_upload = temp_trimmed_path
142         is_trimmed = True
143     except Exception as e:
144         logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
145 full video.")
146
147     # Helper to clean up local temp files
148     def cleanup_temp_files():
149         if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
150             try:
151                 os.remove(temp_trimmed_path)
152             except Exception:
153                 pass
154
155     return video_to_upload, cleanup_temp_files
156
157 def _process_chunked(
158     self,
159     video_id: str,
160     video_to_upload: str,
161     sport_profile,
162     provider,
163     video_duration: float,
164     chunk_duration: float,
165     chunk_overlap: float,
166     ai_scale_width: int,
167     chunks_dir: str,
168     cleanup_temp_files: Callable[[], None],
169 ) -> List[Dict[str, Any]]:
170     """--- CHUNKED PROCESSING --- (long videos): split into overlapping chunks,
171 analyze them in parallel, offset timestamps back to full-video coordinates."""
172     ai_segments: List[Dict[str, Any]] = []
173     step = chunk_duration - chunk_overlap
174     chunk_starts = []
175     t = 0.0
176     while t < video_duration:
177         chunk_starts.append(t)
178         t += step
179     num_chunks = len(chunk_starts)
180     logger.info(
181         f"Video is {video_duration:.1f}s ? splitting into {num_chunks} overlapping
182 chunks "
183         f"({chunk_duration}s each, {chunk_overlap}s overlap)."
184     )
185     os.makedirs(chunks_dir, exist_ok=True)

```

```

183     def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list, dict]:
184         chunk_end = min(chunk_start + chunk_duration, video_duration)
185         actual_chunk_dur = chunk_end - chunk_start
186         chunk_label = f"Chunk {ci+1}/{num_chunks}"
187         chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
188
189         # Extract chunk with ffmpeg
190         logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
191         success = extract_video_chunk(
192             video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
193             reencode=ai_scale_width > 0,
194             scale_width=ai_scale_width if ai_scale_width > 0 else None,
195         )
196         if not success:
197             logger.error(f"Failed to extract {chunk_label}. Skipping.")
198             return chunk_label, [], {}
199
200         # Measure chunk size
201         try:
202             chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
203         except Exception:
204             chunk_size_mb = 0.0
205
206         # Get prompt from sport profile
207         prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
208
209         # Process this chunk through the provider
210         chunk_segments, metrics = provider.analyze_video(
211             chunk_path, prompt, chunk_duration=actual_chunk_dur, label=chunk_label
212         )
213         metrics["file_size_mb"] = chunk_size_mb
214         metrics["segments_found"] = len(chunk_segments)
215
216         # Offset timestamps back to full-video coordinates
217         for segment in chunk_segments:
218             if "start_time" in segment:
219                 try:
220                     segment["start_time"] = float(segment["start_time"]) +
chunk_start
221                 except (ValueError, TypeError):
222                     pass
223             if "end_time" in segment:
224                 try:
225                     segment["end_time"] = float(segment["end_time"]) + chunk_start
226                 except (ValueError, TypeError):
227                     pass
228
229         logger.info(f"{chunk_label} returned {len(chunk_segments)} play segments.")
230
231         # Clean up chunk file
232         try:
233             os.remove(chunk_path)
234         except Exception:
235             pass
236
237         return chunk_label, chunk_segments, metrics
238
239     # Run chunks in parallel. Workers come from indexing.ai_max_workers (same knob
240     # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
241     # gently ramp a cold/prepaid Gemini project, whose burst throttling returns a
242     # misleading "credits depleted" 429 under 5 simultaneous uploads.
243     all_metrics = {}
244     max_workers = max(1, int(self.config.get("indexing", {}).get("ai_max_workers",
245     5)))

```

```

245         logger.info("Starting parallel processing with up to %d concurrent workers...",
max_workers)
246         t_chunking_start = time.time()
247         with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as executor:
248             futures = [executor.submit(process_single_chunk, ci, start) for ci, start in
enumerate(chunk_starts)]
249             for future in concurrent.futures.as_completed(futures):
250                 label, segments, metrics = future.result()
251                 ai_segments.extend(segments)
252                 if metrics:
253                     all_metrics[label] = metrics
254
255             # Oversize refusals must be LOUD: each one is a time range with NO analysis,
256             # so "0 segments / success" here would be a silent failure, not "no rallies".
257             oversize_skips = sum(1 for m in all_metrics.values() if
m.get("oversize_skipped"))
258             if oversize_skips:
259                 logger.error(
260                     f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload cap
and were "
261                     f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time ranges.
"
262                     f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-
encoded/"
263                     f"downscaled before upload."
264                 )
265             run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)
266
267             # generate() give-ups (per-attempt timeout / 503 bursts exhausting retries) are
the SAME
268             # class of silent failure: the chunk's time range gets NO analysis, so "0
segments" there is
269             # "we don't know", not "no rallies". Surface it loudly + count it so a partial
reel isn't
270             # mistaken for a complete one (observed live: a degraded model alias / capacity
503s dropped
271             # whole chunks while the run still "succeeded").
272             generate_fails = sum(1 for m in all_metrics.values() if
m.get("generate_failed"))
273             if generate_fails:
274                 logger.error(
275                     f"{generate_fails}/{num_chunks} chunks FAILED generate (timeout/503
after retries) ? "
276                     f"those time ranges have NO analysis; the result is INCOMPLETE, not 'no
rallies'. "
277                     f"Re-run (transient capacity), or pin a healthy indexing model if the
alias is degraded."
278                 )
279             run_signals.incr(video_id, "generate_failed_chunks", generate_fails)
280
281             # Print Summary Log
282             summary_lines = [
283                 "",
284                 "=" * 60,
285                 "                GEMINI CHUNKING SUMMARY",
286                 "=" * 60,
287                 f" Total time: {time.time() - t_chunking_start:.1f}s",
288                 f" Total chunks: {num_chunks}",
289                 f" Total play segments found: {len(ai_segments)}",
290                 "-" * 60,
291                 f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}",
292                 "-" * 60,
293             ]
294             for i in range(num_chunks):

```



```

295         lbl = f"Chunk {i+1}/{num_chunks}"
296         m = all_metrics.get(lbl, {})
297         sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
298         upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
299         prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
300         gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
301         seg = m.get('segments_found', 0) if m else 0
302         summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} | {gen:<8}
| {seg}")
303     summary_lines.append("=" * 60)
304     logger.info("\n".join(summary_lines))
305
306     # Clean up local temp files
307     cleanup_temp_files()
308     try:
309         os.rmdir(chunks_dir)
310     except Exception:
311         pass
312
313     logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")
314
315     return ai_segments
316
317     def _process_single(
318         self,
319         video_id: str,
320         video_to_upload: str,
321         sport_profile,
322         provider,
323         video_duration: float,
324         cleanup_temp_files: Callable[[], None],
325     ) -> List[Dict[str, Any]]:
326         """--- SINGLE-FILE PROCESSING (short videos) --- analyze the whole video in one
call."""
327         try:
328             file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
329         except Exception:
330             file_sz = 0.0
331
332         # Get prompt from sport profile
333         prompt = sport_profile.get_prompt(chunk_duration=video_duration)
334         ai_segments, metrics = provider.analyze_video(
335             video_to_upload, prompt, chunk_duration=video_duration, label="SingleFile"
336         )
337         if metrics.get("oversize_skipped"):
338             logger.error(
339                 f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and was "
340                 f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no
rallies'."
341             )
342             run_signals.incr(video_id, "oversize_clips_skipped", 1)
343         summary_lines = [
344             "",
345             "=" * 50,
346             "    GEMINI SINGLE FILE SUMMARY",
347             "=" * 50,
348             f" File Size: {file_sz:.1f} MB",
349             f" Upload time: {metrics.get('upload_time', 0):.1f}s",
350             f" API Processing time: {metrics.get('process_time', 0):.1f}s",
351             f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
352             f" Play segments found: {len(ai_segments)}",
353             "=" * 50,
354         ]
355         logger.info("\n".join(summary_lines))

```

```

356         cleanup_temp_files()
357
358         return ai_segments
359
360     def _validate_and_dedupe(self, ai_segments: List[Dict[str, Any]], video_duration:
float) -> List[Dict[str, Any]]:
361         """Parse timestamps, validate/clamp candidates, and resolve overlaps (keep
longer)."""
362         logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
363
364         # 7. Parse timestamps and build candidate segment list
365         idx_cfg = self.config.get("indexing", {})
366         min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
367         max_segment_duration = 90.0 # Singles play segments almost never exceed this
368
369         candidates = []
370         for idx, segment in enumerate(ai_segments):
371             start = _parse_time(segment.get("start_time", 0.0))
372             end = _parse_time(segment.get("end_time", 0.0))
373             if start >= end:
374                 logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
375                 continue
376                 candidates.append({
377                     "idx": idx,
378                     "start": start,
379                     "end": end,
380                     "raw": segment
381                 })
382
383         # 8. Post-hoc validation pass
384         logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
385         validated = []
386         rejected = []
387
388         for c in candidates:
389             label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
390
391             # A: Reject segments that start beyond the video
392             if video_duration > 0 and c["start"] >= video_duration:
393                 rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
394                 continue
395
396             # B: Clamp end_time to video duration
397             if video_duration > 0 and c["end"] > video_duration:
398                 logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
399                 c["end"] = video_duration
400
401             dur = c["end"] - c["start"]
402
403             # C: Reject segments shorter than min_segment_duration
404             if dur < min_segment_duration:
405                 rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
406                 continue
407
408             # D: Flag suspiciously long segments
409             if dur > max_segment_duration:
410                 logger.warning(
411                     f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "

```

```

412         f"This may be a detection error. Keeping but flagging."
413     )
414
415     validated.append(c)
416
417     # E: Resolve overlapping segments (keep the longer one)
418     if len(validated) > 1:
419         # Sort by start time
420         validated.sort(key=lambda r: r["start"])
421         non_overlapping = [validated[0]]
422         for curr in validated[1:]:
423             prev = non_overlapping[-1]
424             # Check if current segment overlaps with the previous one
425             if curr["start"] < prev["end"]:
426                 prev_dur = prev["end"] - prev["start"]
427                 curr_dur = curr["end"] - curr["start"]
428                 kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
429                 kept_label = f"Segment #{kept['idx']+1} [{kept['start']:02f}s -
{kept['end']:02f}s]"
430                 dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:02f}s - {dropped['end']:02f}s]"
431                 rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
432                 if kept == curr:
433                     non_overlapping[-1] = curr # Replace previous with current
434                 else:
435                     non_overlapping.append(curr)
436                 validated = non_overlapping
437
438     # Log validation summary
439     if rejected:
440         reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
441         logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
442         logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")
443
444     stitch_cfg = self.config.get("stitching", {})
445     begin_pad = stitch_cfg.get("begin_padding", 0.5)
446     end_pad = stitch_cfg.get("end_padding", 0.5)
447
448     total_highlight_dur = 0.0
449     for c in validated:
450         total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
451
452     mins = int(total_highlight_dur // 60)
453     secs = int(total_highlight_dur % 60)
454     logger.info(
455         f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
456         f"({total_highlight_dur:01f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
457     )
458
459     return validated
460
461     def _populate_db(self, video_id: str, validated: List[Dict[str, Any]], model_name:
str) -> List[Dict[str, Any]]:
462         """9. Populate SQLite DB with validated play segments; returns segments_data."""
463         segments_data = []
464         for c in validated:
465             start = c["start"]
466             end = c["end"]
467             segment = c["raw"]
468             dur = end - start

```

```

469
470     # Extract shot_count from Gemini or compute estimate if not present
471     shot_count = segment.get("shuttle_exchange_count")
472     if shot_count is None:
473         shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as
fallback estimate
474     else:
475         try:
476             shot_count = int(shot_count)
477         except Exception:
478             shot_count = max(3, int(dur / 0.8))
479
480     # Only store metrics we actually know ? no fabricated data
481     metadata = {
482         "shot_count": shot_count,
483         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
484         "detector": model_name
485     }
486
487     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
488
489     if segment_id:
490         segments_data.append({
491             "id": segment_id,
492             "start_time": start,
493             "end_time": end,
494             "duration": dur,
495             "metadata": metadata
496         })
497
498     return segments_data
499
500     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
501         """
502         Processes a video to detect play segments using Google Gemini Multimodal Video
API,
503         populates SQLite, and indexes detailed events.
504         """
505         model_name = self.config.get("ai_model", self.config.get("indexing",
{})).get("ai_model", "gemini-2.5-flash"))
506
507         # Resolve sport profile + provider (handles the unknown-sport / missing-key /
unknown-provider early returns)
508         resolved = self._resolve_provider(model_name)
509         if resolved is None:
510             return []
511         sport_profile, provider = resolved
512
513         # Optionally trim the video first (debug feature); cleanup removes any temp
file.
514         video_to_upload, cleanup_temp_files = self._maybe_trim_video(video_id,
video_path)
515
516         # 1. Determine actual video duration
517         video_duration = get_video_duration(video_to_upload)
518         if video_duration > 0:
519             logger.info(f"Detected video duration: {video_duration:.1f}s")
520         else:
521             logger.warning("Could not detect video duration. Prompt will not include
duration context.")
522
523         # 2. Decide whether to use chunked processing

```

```

524         idx_cfg = self.config.get("indexing", {})
525         chunk_duration = idx_cfg.get("chunk_duration", 300)      # 5 minutes default
526         chunk_overlap = idx_cfg.get("chunk_overlap", 30)        # 30 seconds overlap
527         use_chunking = video_duration > 0 and video_duration > chunk_duration
528         # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-
copied
529         # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
530         # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.
531         ai_scale_width = int(idx_cfg.get("ai_chunk_scale_width", 1280) or 0)
532
533         # Per-video chunk workdir (concurrency isolation): a concurrent run for another
534         # video keeps its own dir, so cleanup here never removes another run's chunks.
535         chunks_dir = _chunk_workdir(video_id)
536
537         if use_chunking:
538             ai_segments = self._process_chunked(
539                 video_id, video_to_upload, sport_profile, provider, video_duration,
540                 chunk_duration, chunk_overlap, ai_scale_width, chunks_dir,
cleanup_temp_files,
541             )
542         else:
543             ai_segments = self._process_single(
544                 video_id, video_to_upload, sport_profile, provider, video_duration,
cleanup_temp_files,
545             )
546
547         if not ai_segments:
548             return []
549
550         # Populate SQLite DB with parsed play segments
551         validated = self._validate_and_dedupe(ai_segments, video_duration)
552
553         # 9. Populate SQLite DB with validated play segments
554         segments_data = self._populate_db(video_id, validated, model_name)
555
556         return segments_data
557
558     # Register the segmenter
559     segmenter_registry.register(GeminiPlaySegmenter)
560

```

11. user (2026-06-20T08:00:45.869Z)

Found 37 files

```

docs\RALLY_QUALITY_RESEARCH.md
docs\RALLY_DETECTION_QUALITY_REPORT.md
docs\RALLY_DETECTION_GAP_CLOSURE.md
docs\R2_EVAL_METRIC_DESIGN.md
docs\QUALITY_ITERATIONS.md
docs\GOLDEN_VIDEOS.md
docs\CODE_MAP.md
docs\BACKLOG.md
backend\pipeline\segmenters\gemini.py
backend\providers\gemini.py
tests\test_generate_highlights.py
tools\generate_highlights.py
docs\DESKTOP_APP_PLAN.md
docs\VIDEO_RECORDING_GUIDELINES.md
docs\REAL_FOOTAGE_VALIDATION_REPORT.md
docs\PROJECT_DOC_TEMPLATE.md
tests\test_gemini.py
docs\PERSONALIZATION_PLAN.md
backend\reporting\spec.yaml
docs\COMMERCIALIZATION.md
README.md

```

```
backend\eval\fusion_golden.py
docs\STORAGE_SHARING_MODEL.md
docs\GOLDEN_SET_VENDORING.md
tools\cleanup_caches.py
docs\VIDEO_LOCALITY_MODEL.md
tests\test_cleanup_caches.py
backend\utils\video.py
backend\pipeline\validators\relevance.py
backend\pipeline\validators\resolution_fps.py
docs\archives\decisions\DECISIONS.md
docs\UI_PROPOSAL.md
docs\HOSTING_PLAN.md
docs\archives\research\AUDIO_E1_FINDINGS.md
docs\archives\research\RALLY_DETECTION_RESEARCH.md
backend\api\static\style.css
backend\eval\gemini_refine.py
```

12. user (2026-06-20T08:00:46.389Z)

<persisted-output>

Output too large (23.6KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\tool-results\toolu_017YwhhzvjAf3BfdriAKZBs9.txt

Preview (first 2KB):

```
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_async_jobs.py-61-          self.db
= db
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_async_jobs.py-62-
tests\test_async_jobs.py-63-      def process_video(self, video_id, video_path, *a, **k):
tests\test_async_jobs.py:64:          sid = self.db.add_play_segment(video_id, 0.0, 5.0)
tests\test_async_jobs.py-65-          return [{"id": sid, "start_time": 0.0, "end_time": 5.0,
"duration": 5.0, "metadata": {}}]
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_async_jobs.py-66-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_async_jobs.py-67-
--
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_async_jobs.py-70-          self.db
= db
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_async_jobs.py-71-
tests\test_async_jobs.py-72-      def process_video(self, video_id, video_path, *a, **k):
tests\test_async_jobs.py:73:          self.db.add_play_segment(video_id, 9.0, 10.0) # half-
written row pre-crash
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_async_jobs.py-74-          raise
RuntimeError("boom")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_async_jobs.py-75-
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_async_jobs.py-76-
--
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_async_jobs.py-251-      vid_file =
_video(tmp_path)
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_async_jobs.py-252-      vid =
m.compute_video_id(str(vid_file))
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_async_jobs.py-253-      db.add_video(vid, str(vid_file), "processed", [])
tests\test_async_jobs.py:254:      db.add_play_segment(vid, 1.0, 2.0) # prior good result
tests\test_async_jobs.py-255-      mp.setattr(m.registry, "run_all_with_context", lambda *a, **k:
(_fail(), {}))
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_async_jobs.py-256-
...
</persisted-output>
```

13. assistant (2026-06-20T08:00:51.916Z)

Now let me check the sports prompt schema (what fields Gemini emits), the motion segmenter's events/metadata, and confirm whether any frame/thumbnaill extraction exists for playback. Let me also check the UI proposal and static UI for what already exists.

14. user (2026-06-20T08:00:52.769Z)

```
1      from backend.sports.base import SportProfile
2
3      class BadmintonSportProfile(SportProfile):
4          @property
5          def name(self) -> str:
6              return "badminton"
7
8          def get_prompt(self, chunk_duration: float = 0.0) -> str:
9              base_prompt = ""System Role & Objective
10             You are a highly precise computer vision data extraction engine specializing in
11             badminton match analysis. Your exclusive objective is to parse a full-length match video and
12             generate a flawless JSON array containing exact rally boundaries and shot metrics for automated
13             highlight generation.
14
15             Core Directives
16             Detect every single active rally in the video. Do not miss any live gameplay.
17             Extract high-precision timestamps as TOTAL DECIMAL SECONDS from the start of the video
18             for exact frame-accurate cuts.
19             Estimate the number of shots exchanged (net crossings) and provide a confidence score
20             for that count, acknowledging frame-sampling limitations.
21
22             CRITICAL TIMESTAMP FORMAT RULES:
23             - All start_time and end_time values MUST be expressed as TOTAL SECONDS from the
24             beginning of the video, as a decimal number.
25             - Example: A rally starting at 1 minute 14 seconds into the video MUST be written as
26             74.0, NOT as "1:14" or 114.
27             - Example: A rally starting at 2 minutes 30.5 seconds MUST be written as 150.5, NOT as
28             "2:30.5" or 230.5.
29             - NEVER use MM:SS or HH:MM:SS colon-separated format. NEVER concatenate minutes and
30             seconds into a single number without converting.
31             - The downstream pipeline will BREAK if timestamps are not in total decimal seconds.
32
33             Positive Constraints: Rally Boundaries
34             Start Condition: The exact fractional second the server's racket makes contact with the
35             shuttlecock.
36             End Condition: The rally concludes ONLY when both of the following physical criteria are
37             met sequentially: First, the shuttlecock hits the court surface, strikes the net and drops, or
38             lands visibly out of bounds. Second, a 0.3 to 0.5-second buffer has elapsed where both players
39             have visibly ceased all kinetic momentum (no lunging, sliding, or racket follow-through).
40
41             Negative Constraints: Anomaly Rejection (CRITICAL)
42             Replay Rejection: You must strictly ignore and filter out all slow-motion instant
43             replays. Replays are not live rallies.
44             Dead-Time Rejection: Ignore all Hawkeye/line-call challenges, interval coaching, towel
45             breaks, court mopping, changing shuttles, and player celebrations.
46             Pre-serve Rituals: Do not start the timestamp during the players' setup or stance. Wait
47             for physical racket-to-shuttle contact.
48
49             Shot Tracking & Data Confidence
50             Shot Definition: Count every successful traverse of the shuttlecock across the net
51             (serves, clears, smashes, drops, drives). Do not count the final grounding or pre-net faults.
52             Confidence Scoring: Because the shuttlecock moves at extreme velocities that may fall
53             between sampled frames, provide a shot_count_confidence metric (Low, Medium, High). Base your
54             estimate on player biomechanics, positioning, and racket swings if the shuttle itself is
55             blurred.
56
57             Strict Output Schema
58             Output strictly in the following JSON array format. Do not include markdown formatting,
59             introductory text, conversational filler, or explanations. Any deviation from this schema will
60             break the downstream pipeline.
61             Remember: start_time and end_time are TOTAL SECONDS (decimal), not MM:SS.
62
63             [
```

```

42     {
43         "start_time": 74.0,
44         "end_time": 81.3,
45         "shuttle_exchange_count": 11,
46         "shot_count_confidence": "High"
47     },
48     {
49         "start_time": 150.5,
50         "end_time": 174.2,
51         "shuttle_exchange_count": 24,
52         "shot_count_confidence": "Medium"
53     }
54 ]
55 """
56     if chunk_duration > 0.0:
57         duration_context = (
58             f"IMPORTANT VIDEO CONTEXT: This video segment is exactly
{chunk_duration:.1f} seconds long. "
59             f"All timestamps MUST be between 0 and {chunk_duration:.1f}. "
60             f"Any timestamp beyond {chunk_duration:.1f}s is INVALID.\n\n"
61         )
62         return duration_context + base_prompt
63     return base_prompt
64
65     def get_relevance_config(self) -> dict:
66         return {
67             "court_green_lower": [35, 40, 40],
68             "court_green_upper": [85, 255, 255],
69             "court_pixels_min_ratio": 0.05
70         }
71

```

15. user (2026-06-20T08:00:53.265Z)

```

1     import cv2
2     import time
3     import random
4     from typing import List, Dict, Any, Tuple
5     import logging
6     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
7     from backend.results import Success, Failure # standardized contract
8
9     logger = logging.getLogger(__name__)
10
11     class MotionPlaySegmenter(BasePlaySegmenter):
12         @property
13         def name(self) -> str:
14             return "motion"
15
16         @property
17         def description(self) -> str:
18             return "Traditional CV pixel-motion heuristics tracking frame differences."
19
20         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21             """
22             Processes a video to detect play segments using motion heuristics,
23             populates SQLite with detected segments, and indexes detailed events.
24             """
25             if not player_pool:
26                 player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28             cap = cv2.VideoCapture(video_path)
29             if not cap.isOpened():
30                 logger.error(f"Segmenter could not open video: {video_path}")

```



```

31         return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33         final_segments = self._detect_motion_segments(cap, max_frames)
34
35         segments_data = self._populate_db_with_events(video_id, final_segments,
player_pool)
36
37         return Success(value=segments_data)
38
39     def _detect_motion_segments(self, cap: Any, max_frames: int = None) ->
List[Tuple[float, float]]:
40         """Run the pixel-motion heuristic over ``cap`` and return the detected
41         (start, end) play segments.
42
43         Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
44         thresholds it into raw segments, then merges dead-time gaps and filters
45         out segments shorter than ``min_segment_duration``. ``cap`` is released
46         before returning. This is the detection half of ``process_video`` ? it
47         performs no DB writes and no fabrication.
48         """
49         fps = cap.get(cv2.CAP_PROP_FPS)
50         total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
51         duration = total_frames / fps if fps > 0 else 0
52
53         # Sub-sample settings (5 FPS for analysis)
54         analysis_fps = 5.0
55         frame_interval = max(1, int(fps / analysis_fps))
56
57         idx_cfg = self.config.get("indexing", {})
58         motion_threshold = idx_cfg.get("motion_threshold", 0.08)
59         min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
60         dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
61
62         prev_gray = None
63         motion_signals = []
64         timestamps = []
65
66         frame_idx = 0
67         processed_count = 0
68         t_start = time.time()
69         # Emit a progress line roughly every 5% of the video so long runs are
70         # observable instead of silent (a full 1080p match is ~tens of minutes).
71         progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
72         next_progress = progress_step
73         logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
74                 f"({~{duration/60:.1f} min), analyzing every {frame_interval}th
frame")
75         while True:
76             # Skip decoding for intermediate frames using cap.grab() for high
performance
77             for _ in range(frame_interval - 1):
78                 if not cap.grab():
79                     break
80                 frame_idx += 1
81
82             ret, frame = cap.read()
83             if not ret:
84                 break
85
86             # Process frame
87             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
88             # Apply Gaussian Blur to smooth noise
89             gray = cv2.GaussianBlur(gray, (21, 21), 0)
90

```

```

91         t = frame_idx / fps
92         timestamps.append(t)
93
94         if prev_gray is not None:
95             # Frame absolute difference (highly optimized motion tracking)
96             diff = cv2.absdiff(prev_gray, gray)
97             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
98
99             # Dilate to fill gaps
100            thresh = cv2.dilate(thresh, None, iterations=2)
101
102            # Motion ratio
103            motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
104            motion_signals.append(motion_ratio)
105        else:
106            motion_signals.append(0.0)
107
108        prev_gray = gray
109        frame_idx += 1
110        processed_count += 1
111
112        if progress_step and frame_idx >= next_progress:
113            pct = 100.0 * frame_idx / total_frames
114            elapsed = time.time() - t_start
115            eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
116            logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
"
117                      f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
~{eta:.0f}s left")
118            next_progress += progress_step
119
120        if max_frames is not None and processed_count >= max_frames:
121            logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
122            break
123
124        if frame_idx >= total_frames:
125            break
126
127    cap.release()
128
129    # Smooth signal using a moving average
130    window_size = 5
131    smoothed = []
132    for i in range(len(motion_signals)):
133        start = max(0, i - window_size // 2)
134        end = min(len(motion_signals), i + window_size // 2 + 1)
135        smoothed.append(sum(motion_signals[start:end]) / (end - start))
136
137    # Detect active play boundaries
138    in_segment = False
139    segment_start = None
140    raw_segments: List[Tuple[float, float]] = []
141
142    for i, t in enumerate(timestamps):
143        signal = smoothed[i]
144        if signal > motion_threshold:
145            if not in_segment:
146                in_segment = True
147                segment_start = t
148        else:
149            if in_segment:
150                in_segment = False
151                raw_segments.append((segment_start, t))

```

```

152
153     # Handle video ending while in active segment
154     if in_segment:
155         raw_segments.append((segment_start, timestamps[-1]))
156
157     # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
158     merged_segments: List[Tuple[float, float]] = []
159     if raw_segments:
160         curr_start, curr_end = raw_segments[0]
161         for next_start, next_end in raw_segments[1:]:
162             if next_start - curr_end < dead_time_merge_gap:
163                 # Merge
164                 curr_end = next_end
165             else:
166                 merged_segments.append((curr_start, curr_end))
167                 curr_start, curr_end = next_start, next_end
168         merged_segments.append((curr_start, curr_end))
169
170     # Post-Processing: 2. Filter segments shorter than min_segment_duration
171     final_segments: List[Tuple[float, float]] = [
172         r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
173     ]
174
175     return final_segments
176
177 def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]],
178                             player_pool: List[str]) -> List[Dict[str, Any]]:
179     """Persist ``final_segments`` and their fabricated metadata/events to the DB.
180
181     For each detected segment this assigns a random server/receiver, fabricates
182     rich tags, then indexes a serve / 0?3 smash / termination event set ? all
183     with the stdlib ``random`` module (the legacy demo's fabricated baseline).
184     Returns the per-segment dicts handed back to the caller. The ``random``
185     call order here is load-bearing for reproducibility and must not change.
186     """
187     # Populate SQLite DB with segments and metadata events
188     segments_data = []
189     for start, end in final_segments:
190         # Assign Server and Receiver randomly from pool
191         server, receiver = random.sample(player_pool, 2)
192
193         # Formulate dynamic rich tags
194         dur = end - start
195         metadata = {
196             "shot_count": random.randint(4, 25),
197             "intensity": "high" if dur > 12 else "medium",
198             "avg_speed_kmh": round(random.uniform(40, 85), 1)
199         }
200
201         segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)
202
203         if segment_id:
204             # Add sub-segment events
205             # A: Serve Event (always at start)
206             serve_foul = random.random() < 0.15 # 15% chance of serve foul
207             self.db.add_event(
208                 segment_id=segment_id,
209                 timestamp=start + 0.5,
210                 event_type="serve",
211                 player=server,
212                 details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
213             )

```

```

214
215         # B: Smash Events (1 to 3 random smashes per segment)
216         smashes_count = random.randint(0, 3)
217         for _ in range(smashes_count):
218             smash_time = random.uniform(start + 1.5, end - 1.0)
219             smasher = random.choice([server, receiver])
220             self.db.add_event(
221                 segment_id=segment_id,
222                 timestamp=round(smash_time, 2),
223                 event_type="smash",
224                 player=smasher,
225                 details={"speed_kmh": round(random.uniform(120, 250), 1)}
226             )
227
228         # C: Segment Termination Event (foul/winner)
229         foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
230         is_foul = random.random() > 0.4
231         error_player = random.choice([server, receiver])
232
233         self.db.add_event(
234             segment_id=segment_id,
235             timestamp=end - 0.2,
236             event_type="foul" if is_foul else "winner",
237             player=error_player,
238             details={"type": random.choice(foul_types)} if is_foul else
239             {"shot_type": "drop" if random.random() > 0.5 else "smash"}
240         )
241
242         # Store segment representation to return to caller
243         segments_data.append({
244             "id": segment_id,
245             "start_time": start,
246             "end_time": end,
247             "duration": dur,
248             "server": server,
249             "receiver": receiver,
250             "metadata": metadata
251         })
252
253         return segments_data
254
255     # Register the segmenter
256     segmenter_registry.register(MotionPlaySegmenter)

```

16. user (2026-06-20T08:00:55.206Z)

```

backend\pipeline\detectors\wasb_infer.py:71:      """Best-effort integer frame id from a frame
filename (e.g. frame_000180.png -> 180)."""
backend\pipeline\detectors\wasb_infer.py:208:_STREAM_FRAME_FMT = "{:08d}.png"
backend\pipeline\detectors\wasb_infer.py:214:      It mirrors the on-disk names ``extract_frames``
writes (``{i:08d}.png``) so the
backend\pipeline\detectors\wasb_infer.py:296:      The disk path is ``frame_bgr ->
cv2.imwrite(PNG) -> Image.open(PNG).convert('RGB')``;
backend\pipeline\detectors\wasb_infer.py:369:      globbed (``*.png``/``*.jpg``) and
``ImageDataset`` reads each file via ``cv2.imread``.
backend\pipeline\detectors\wasb_infer.py:375:      to the disk round-trip (``cv2.imwrite`` PNG
-> ``Image.open().convert('RGB')`` ==
backend\pipeline\detectors\wasb_infer.py:407:      frames =
sorted(glob.glob(osp.join(frames_dir, "*.png")) + glob.glob(osp.join(frames_dir, "*.jpg")))
backend\pipeline\detectors\wasb_infer.py:547:def extract_frames(video_path: str, frames_dir:
str) -> str:
backend\pipeline\detectors\wasb_infer.py:548:      """Decode a video to PNG frames named by source
frame index (00000000.png ...).
backend\pipeline\detectors\wasb_infer.py:561:      existing = len(glob.glob(osp.join(frames_dir,

```

```

"*.png"))
backend\pipeline\detectors\wasb_infer.py:571:         cv2.imwrite(osp.join(frames_dir,
f"{i:08d}.png"), frame)
backend\pipeline\detectors\wasb_infer.py:596:         returns exactly the BGR uint8 array that
`cv2.imwrite`/`cv2.imread` of a PNG would
backend\pipeline\detectors\wasb_infer.py:689:         frames_dir = extract_frames(args.video,
args.frames_out_dir or (args.video + "_frames"))

```

17. assistant (2026-06-20T08:01:00.127Z)

The frame extraction (`extract\_frames` in wasb_infer.py) is a transient WSL-side intermediate for the detector pass ? PNG frames named by index, deleted on success (cache hygiene), not per-rally thumbnails for a UI. Let me confirm there is no proxy/thumbnail in the API/static UI and check the UI proposal doc and what `query\_play\_segments` returns to a UI, plus verify the candidate_segments full column list and check whether game/set/score columns exist anywhere.

18. user (2026-06-20T08:01:01.079Z)

```

1      # UI Proposal ? khelsutra.guru web app (alpha ? beta)
2
3      **Status:** PROPOSAL ? awaiting owner ratification (no implementation starts without
explicit approval, per CLAUDE.md).
4      **Date:** 2026-06-10 . **Inputs:** PMF_MARKET_RESEARCH, COMMERCIALIZATION,
PLATFORM_ARCHITECTURE, DEFERRED_HARDENING_PLAN (#16), ROADMAP, VIDEO_RECORDING_GUIDELINES,
reporting/spec.yaml, current API + static UI.
5      **Companion:** `docs/HOSTING_PLAN.md` (domain, topology, costs).
6
7      ## 1. Strategic frame ? why a UI now, and what it secretly is
8
9      The UI is not just product surface; it is the **corpus-growth engine**. The single
highest-leverage
10     item in NEXT_STEPS is growing the golden corpus, and the v1 scope already commits to a
11     **human correction loop**. Every tester who accepts/rejects/nudges a rally boundary in
the review
12     screen produces exactly the labels the Gen-0/M0.x track is starved for (subject to the
per-user
13     training opt-in guardrail). 10?25 alpha testers ? a steady stream of labeled, real-
world,
14     multi-court-biased footage ? the distribution where the owned model's 0.66 ship target
lives.
15
16     Serving economics per the 2026-06-10 quality table: alpha serves the **Gemini path**
17     (0.93 on clean footage, ~$0.05/run) ? no GPU required in the request path. The
WASB/owned-model
18     path stays on the owner's box and is *not* exposed in the alpha UI (segmenter choice
hidden).
19
20     **The CLI is not replaced.** The web app talks to the same FastAPI surface the CLI
drives; the CLI
21     remains the power-user/ops tool.
22
23     ## 2. Personas (from PMF research)
24
25     - **Coach / academy (primary, B2B, India):** batches of match recordings, wants rally
review,
26     per-player reels for students/parents, per-rally stats. Already records on static
cameras.
27     - **Enthusiast (B2C funnel):** records own matches, wants a highlight reel + basic
stats, shares
28     with friends. Tripod discipline is the known adoption risk ? the UI must teach
recording
29     guidelines *before* upload, not fail videos after.
30
31     ## 3. ALPHA ? "quick & dirty" (goal: 10?25 invited testers, end-to-end, zero hand-
holding)

```

```

32
33     **Stack:** React + Vite + Tailwind SPA, deployed per HOSTING_PLAN (static hosting, $0).
34     API = existing FastAPI on the GPU box via tunnel at `api.khelsutra.guru`.
35     **Auth:** Cloudflare Access email-allowlist (OTP), free ?50 users, **zero auth code** ?
backend
36     reads the authenticated-email header for per-user scoping (alpha-grade tenancy: header
is
37     trustworthy because the tunnel only accepts Access-authenticated traffic).
38
39     ### Alpha screens (6)
40
41     | # | Screen | What it does | Backed by |
42     |---|-----|-----|-----|
43     | A1 | **Matches** (home) | User's videos, status chip per video (verdict ?/??/?),
upload CTA | `videos` + owner scoping (B4) |
44     | A2 | **Upload & Process** | Drag-drop; inline recording-guidelines checklist (pre-
upload education from VIDEO_RECORDING_GUIDELINES); chunked upload; then auto-starts processing |
B2 upload endpoint ? B1 job |
45     | A3 | **Job progress** | Stage list (validation ? segmentation ? AI confirm ? report)
with live status; verdict banner on completion | B1 `GET /api/jobs/{id}` poll |
46     | A4 | **Rally review** ? | Video player + rally list; click rally ? seek & play that
span; per-rally **Accept / Reject / nudge start?end ±0.5 s**; bulk "all good"; chips: duration,
shot count (+confidence) | B6 correction endpoints ? `candidate_segments.human_label/notes` ?
**the corpus engine** |
47     | A5 | **Highlights** | Select accepted rallies ? name reel ? compile ? download; list
of past reels | `/api/compile` + B3 download endpoint |
48     | A6 | **Report** | Renders the customer-audience report (verdict, notes, FAQ refs);
"nerd view" toggle shows technical.md | existing reporting artifacts |
49
50     ### Backend work the alpha actually requires (honest dependency list)
51
52     | ID | Item | Notes |
53     |---|-----|-----|
54     | B1 | **Async job model ? DEFERRED_HARDENING_PLAN #16** | The committed next PLATFORM
slice; `POST /api/process` ? 202 + `job_id`, `jobs` table (user_version 1?2),
ThreadPoolExecutor, `GET /api/jobs/{id}`. **Approving this proposal = greenlighting #16.** A
5?60 min blocking HTTP call cannot sit behind a tunnel UI. |
55     | B2 | **Chunked upload endpoint** | `POST /api/upload` accepting ?90 MB parts (free-
tier proxy body limit is 100 MB), assemble ? MD5 ? register under `allowed_video_root/<user>/`.
Size cap (e.g. 4 GB) + mp4/mov allowlist. |
56     | B3 | **Download endpoint** | `GET /api/highlights/{video_id}/{name}` streaming the
compiled reel (today `/api/compile` returns a server-local path the browser can't reach). |
57     | B4 | **Per-user scoping** | `owner_email` column on `videos` (migration), read from
the Access header, filter every query. Alpha-grade, single DB. |
58     | B5 | **Retire/replace static UI** | The SPA supersedes `backend/api/static` (which has
a known `rallies` vs `play_segments` field bug ? don't fix, replace). |
59     | B6 | **Correction-loop endpoints** | `PATCH /api/segments/{id}` ?
accept/reject/boundary-nudge persisted to `candidate_segments.human_label`/`notes` (+
provenance: who, when). This is v1-scope "correction loop" made real. |
60     | B7 | **Exposure hardening** | CORS narrowed to the app origin; basic rate limit;
upload quota per user; `allowed_video_root` set; Gemini key rotated (already in NEXT_STEPS
housekeeping). |
61
62     ### Alpha non-goals (explicit)
63
64     No payments, no orgs/teams, no athlete profiles, no share links, no owned-model serving,
no
65     multi-sport selector (badminton only), no notifications, no mobile-perfect layouts
(usable on
66     phone, optimized for laptop), no conversational QA (deferred in roadmap).
67
68     ### Alpha effort estimate
69
70     B1 ? 2?3 sessions · B2?B7 ? 2?3 sessions · SPA (6 screens) ? 3?4 sessions ? **~8?10
working

```

71 sessions** to first tester invite. Sequencing: B1 ? B2/B3/B4 ? SPA skeleton + A2/A3 ? A4
 ? A5/A6 ? B7 ? invite.

72

73 ## 4. BETA ? polished (goal: paying pilot academies)

74

75 Maps deliberately onto PLATFORM_ARCHITECTURE P1?P3 ? the UI roadmap and the platform
 roadmap are

76 the same roadmap.

77

78 1. **Drop 1 ? Accounts & academies (P1):** real auth (Firebase Auth ? GCP-native, free
 tier ? or

79 Clerk), academy workspace, coach invites athletes (**18+ only**, per guardrail),
 roles

80 (owner/coach/athlete). Coach?athlete stays an **optional overlay** on the generic
 schema.

81 2. **Drop 2 ? Storage & worker (P2/P3):** direct-to-cloud resumable uploads (signed URLs
 ? bypasses

82 proxy body limits entirely), reels served from object storage; control plane moves to
 Cloud Run;

83 **GPU box becomes the first pull-based `byo\_worker`** (outbound-only, exactly P3's
 security

84 model). Email notification when a reel is ready.

85 3. **Drop 3 ? Review v2:** keyboard shortcuts (J/K/L, [/] nudge), frame-step, timeline
 minimap with

86 rally bars, batch accept, rally tags, athlete-facing **expiring share links**, per-
 player

87 aggregates and match-over-match trends (v1 stats only ? **no gait/biometrics**, GDPR
 Art. 9).

88 4. **Drop 4 ? Monetization & consent:** Razorpay per-match credits (INR; PMF: B2C
 \$60?180/yr,

89 academy \$300?1,500/yr equivalents, tuned per-match for India), free-tier match
 allowance;

90 **training-data opt-in**: per-user toggle default OFF + per-match consent record +
 adult-only

91 attestation at upload (guardrails: minors excluded from training pool; separate
 explicit opt-in).

92

93 Beta effort: ~4?6 weeks part-time across the four drops; each drop independently
 shippable.

94

95 ## 5. Decisions needed from owner

96

97 1. **Greenlight** alpha scope above (= greenlights DEFERRED_HARDENING_PLAN #16)?

98 2. **RESOLVED 2026-06-10:** product name = **KhelSutra Guru**; domain =
 kheelsutra.guru (GoDaddy receipt, 3-yr registration + domain protection, renews 2029-06).

99 3. Per-user **training opt-in default OFF** ? confirm (recommended; conservative per
 guardrails).

100 4. Alpha invite list: the C13 interview academies + camera-pilot contacts?

101 5. SPA stack confirm: React + Vite + Tailwind (proposed) vs anything you prefer.

102 6. **RESOLVED 2026-06-10:** separate **private repo `kheelsutra`** (owner-ratified ? the
 product's

103 "heart and brain"): SPA in `web/`, control plane joins at beta; backend PRs B1?B7
 stay in the

104 indexer repo. Keeps the indexer's licensing/test discipline untouched.

105

106 *Once ratified, decisions 1?6 get recorded in `docs/archives/decisions/DECISIONS.md` per
 convention.*

107

19. user (2026-06-20T08:01:02.908Z)

backend\annotations__init__.py:4:harness scores ? so golden-set labels can come from a file, a
 human vendor, or an

backend\annotations\base.py:60: than returning an empty list (which would silently skew
 downstream scores).

```

backend\config\models.py:126:    detector_score_threshold: float = 0.5
backend\eval\annotations.py:49:    rally-annotator ``rally_number,start_time,end_time,...``
schema. Strict: raises
backend\eval\annotations.py:51:    surface immediately rather than silently skewing scores.
Returns intervals sorted by
backend\eval\experiment.py:9:    - `backend.eval.metrics`          ? `score`, `match_intervals`,
`interval_iou`
backend\eval\experiment.py:52:from backend.eval.metrics import Interval, match_intervals, score
backend\eval\experiment.py:57:from backend.eval.score_calibration import fit_isotonic, fit_platt
# C2: calibrate cue scores
backend\eval\experiment.py:70:    score an unlabeled video with no pinned model, or a gate
referencing an absent
backend\eval\experiment.py:99:    `compare()` scores videos with the SHIPPED model as-is (no
per-fold training);
backend\eval\experiment.py:305:    is nothing to score with. ``threshold``/``calibrator`` (both
fitted on the TRAINING
backend\eval\experiment.py:351:    f1s = [score(predict(variant, vc, model=model,
threshold=t, calibrator=calibrator),
backend\eval\experiment.py:386:    - **learned, pinned model** (`classifier_model` set):
score every video with the
backend\eval\experiment.py:424:    sc = score(preds, vc.gts, iou)
backend\eval\experiment.py:442:    """Per-metric mean + bootstrap CI over the per-video scores
(C1 ? never a bare mean)."""
backend\eval\experiment.py:490:    The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
backend\eval\experiment.py:491:    learned variants are scored LOVO-honestly. The result is a
self-contained record fit for
backend\eval\calibrate_local.py:10:videos we use leave-one-video-out (tune on the others, score
the held-out one) ?
backend\eval\calibrate_local.py:95:    # Honest cross-video: pick config on the OTHER videos,
score the held-out one.
backend\eval\calibrate_local.py:97:    # Baseline's held-out score (no tuning) for the honest
before/after.
backend\eval\calibration.py:6:1. """Did precision improve, by how much?""" ?
`improvement_report` scores a
backend\eval\calibration.py:13:    train folds, score on the held-out fold. The
**generalization_gap**
backend\eval\calibration.py:16:    pick the config on all *other* videos, score on the held-out
video. Reports
backend\eval\calibration.py:17:    per-video held-out scores + mean±std. This is the number to
trust.
backend\eval\calibration.py:28:from backend.eval.metrics import score, Score, Interval
backend\eval\calibration.py:43:    return score(preds, gts, iou_threshold)
backend\eval\calibration.py:66:    val = getattr(score(predict_fn(cfg), gts, iou_threshold),
metric)
backend\eval\calibration.py:76:    Per fold: choose the best config on the train rallies, score
it on the held-out
backend\eval\calibration.py:85:    `leave_one_video_out` (each video scored whole) or
`improvement_report` on a
backend\eval\calibration.py:91:    train_scores, test_scores, picked = [], [], []

```

[Showing results with pagination = limit: 30]

20. assistant (2026-06-20T08:01:09.422Z)

The grep confirms there are no `game\_index`, `set\_index`, `score`, `player\_identity`, or `shot\_type` columns anywhere in backend code ? the only "score" hits are eval-metric F1 scoring, not match score. The `detector\_score\_threshold` is a config knob, not a per-rally field.

Let me confirm the full candidate_segments column set (the `human\_label`/`notes`/`confirmed\_segment\_id` correction-loop columns) is complete, and verify there's no proxy video generation and how the static UI plays rallies. Let me check the static UI and the candidate_segments columns once more, plus look at `query\_play\_segments` output for what a UI gets.

21. user (2026-06-20T08:01:10.089Z)

```
backend\api\static\index.html
backend\api\static\style.css
backend\api\static\app.js
```

22. user (2026-06-20T08:01:11.896Z)

```
backend\api\static\index.html:110:                <p>Enter a local video filepath in
the sidebar and run indexing to build the database index.</p>
backend\api\static\app.js:392:                const dlUrl =
`/api/highlights/${currentVideoId}/${encodeURIComponent(outputName)}`;
```

23. user (2026-06-20T08:01:12.754Z)

```
backend\main.py:332:def query_play_segments(req: QueryRequest):
backend\infrastructure\database.py:393:     def query_candidate_segments(self, video_id: str,
detector: Optional[str] = None,
backend\infrastructure\database.py:418:     def get_video(self, video_id: str) ->
Optional[Dict[str, Any]]:
backend\infrastructure\database.py:427:     def query_play_segments(self, video_id: str, filters:
Dict[str, Any]) -> List[Dict[str, Any]]:
backend\utils\video.py:7:def get_video_duration(video_path: str) -> float:
```

24. assistant (2026-06-20T08:01:20.084Z)

Let me check the static UI's app.js to confirm how rallies are shown today (no per-rally clip playback / no proxy), and confirm the `extract\_video\_chunk` util is the only clip-cut path. Let me look at the relevant portion of app.js.

25. user (2026-06-20T08:01:21.410Z)

```
300                <svg width="48" height="48" fill="none" stroke="currentColor" stroke-
width="1.5" viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" d="M3.75
5.25h16.5m-16.5 4.5h16.5m-16.5 4.5h16.5m-16.5 4.5h16.5"/></svg>
301                <h3>No Rallies Available</h3>
302                <p>${msg}</p>
303            </div>
304        `;
305    };
306
307    const formatTime = (seconds) => {
308        const m = Math.floor(seconds / 60);
309        const s = Math.floor(seconds % 60);
310        const ms = Math.floor((seconds % 1) * 10);
311        return `${m.toString().padStart(2, '0')}:${s.toString().padStart(2,
'0')}.${ms}`;
312    };
313
314    // --- 4. RALLY TIMELINE FILTER QUERY ---
315    btnQuery.addEventListener("click", async () => {
316        if (!currentVideoId) {
317            alert("Please process a video first before running filters.");
318            return;
319        }
320
321        const server = queryServer.value;
322        const receiver = queryReceiver.value;
323        const duration = parseFloat(queryDuration.value) || null;
324        const event = queryEvent.value;
325
326        showLoader("Filtering Database Index", "Fetching rallies matching exact
relational conditions...");
327
```

```

328     try {
329         const res = await fetch("/api/query", {
330             method: "POST",
331             headers: { "Content-Type": "application/json" },
332             body: JSON.stringify({
333                 video_id: currentVideoId,
334                 server: server || null,
335                 receiver: receiver || null,
336                 duration_min: duration || null,
337                 event_type: event || null
338             })
339         });
340
341         if (!res.ok) throw new Error("Failed to filter index.");
342
343         const data = await res.json();
344         renderRallyCards(data.rallies || []);
345     } catch (err) {
346         alert(`Query failed: ${err.message}`);
347     } finally {
348         hideLoader();
349     }
350 });
351
352 // --- 5. COMPILER BOARD MANAGEMENT ---
353 const updateCompilerBar = () => {
354     const count = selectedRallyIds.size;
355     if (count > 0) {
356         compileBoard.style.display = "flex";
357         compileSelectedCount.textContent = `${count} Rally Segment${count > 1 ? 's'
: ''} Selected`;
358     } else {
359         compileBoard.style.display = "none";
360     }
361 };
362
363 btnCompile.addEventListener("click", async () => {
364     if (!currentVideoId || selectedRallyIds.size === 0) {
365         alert("No rallies selected for compiling highlights.");
366         return;
367     }
368
369     const outputName = highlightOutputName.value.trim() ||
"highlights_compiled.mp4";
370     showLoader("Compiling Highlight Reels", "Triggering localized high-speed FFmpeg
stitching demuxers...");
371
372     try {
373         const res = await fetch("/api/compile", {
374             method: "POST",
375             headers: { "Content-Type": "application/json" },
376             body: JSON.stringify({
377                 video_id: currentVideoId,
378                 // API field is segment_ids (the old rally_ids never matched the
request model)
379                 segment_ids: Array.from(selectedRallyIds),
380                 output_filename: outputName
381             })
382         });
383
384         if (!res.ok) {
385             const err = await res.json();
386             throw new Error(err.detail || "Concatenation failed.");
387         }
388

```

```

389         await res.json();
390         // B3 (PR-2): offer the reel via the streaming download endpoint instead of
391         // alerting a server-local path the browser can't reach.
392         const dlUrl =
393         `/api/highlights/${currentVideoId}/${encodeURIComponent(outputName)}`;
394         let dl = document.getElementById("highlight-download-link");
395         if (!dl) {
396             dl = document.createElement("a");
397             dl.id = "highlight-download-link";
398             dl.style.cssText = "display:block;margin-top:10px;font-weight:bold;";
399             compileBoard.appendChild(dl);
400         }
401         dl.href = dlUrl;
402         dl.setAttribute("download", outputName);
403         dl.textContent = `? Download ${outputName}`;
404         alert(`Highlight compilation completed! Use the download link to save
405         ${outputName}.`);
406     } catch (error) {
407         alert(`Stitching compilation failed: ${error.message}`);
408     } finally {
409         hideLoader();
410     }

```

26. assistant (2026-06-20T08:01:28.297Z)

I now have a complete picture. The current static UI has no ``<video>`` player element and no per-rally clip playback ? it renders rally cards (with a ``data.rallies`` field bug, since the API returns ``play_segments``), lets the user select segment IDs, and compiles/downloads a single reel. There is no proxy video and no per-rally thumbnail. Let me do a final confirmation grep to be certain no ``score`/`game`/`set`` column or proxy generation exists, and confirm the candidate_segments has no game/score columns by re-reading the v1 migration (already done). I have everything needed.

27. user (2026-06-20T08:02:35.575Z)

Structured output provided successfully

28. assistant (2026-06-20T08:02:49.304Z)

I've mapped the complete data contract. Summary of what the code actually does:

```

**DB schema (backend/infrastructure/database.py, PRAGMA user_version ladder 0?5):**
- `play_segments` is the confirmed-rally table a selection UI consumes today: `id, video_id,
start_time, end_time, duration, server, receiver, metadata(JSON)`. The default `gemini`
segmenter writes `metadata = {shot_count, shot_count_confidence, detector}`; `server`/`receiver`
are set ONLY by the `motion` demo and are random fabrications.
- `candidate_segments` (v1) holds raw detections plus the dead correction-loop columns
`human_label`, `notes`, `confirmed_segment_id` ? schema exists, **no code path writes them**.
- `events` (serve/smash/foul/winner) are written ONLY by `motion` and are entirely `random.`
fabricated.
- `jobs` (v2/v3), `user_models` (v5) round out the ladder.

```

```

**Fields TODAY vs roadmap:** real per-rally fields = id, start/end/duration, video_id, and
Gemini's shot_count(+confidence). **Absent with no detection at all:** shot_type, real player
identity, match/point score, game/set index, and per-confirmed-rally confidence. `play_segments`
has no score/game/confidence columns; grep confirms the only "score" in backend is eval-F1 and a
`detector_score_threshold` config knob.

```

```

**Reel compose:** `POST /api/compile` takes `{video_id, segment_ids[], output_filename}` ?
resolves `videos.filepath` + the selected `play_segments` time-pairs ?
`VideoCompiler.compile_highlights(raw_path, [(start,end)], name)` ? de-dupes overlaps, per-clip
re-encode (libx264/superfast/crf22 + optional watermark) ? concat `-c copy` ? single MP4 in
`output_dir`. Returns `{status, filepath}`; the browser downloads via `GET`

```

/api/highlights/{video_id}/{filename}`. No summary sidecar at compile time.

****Playback:**** No low-res proxy and no per-rally thumbnails/frames exist or are persisted. The only frame extraction is the transient WSL detector cache (deleted on success). No endpoint streams an arbitrary `[start,end]` span or the original source ? only compiled reels are downloadable. The current static UI has no `` element and a known `data.rallies` field bug; a real review UI would need a new source-serving or per-span clip/proxy endpoint that does not exist yet (UI_PROPOSAL B5/B6 are proposed, not built).

Full structured schema returned via StructuredOutput.